

OPC UA Communication Engine — Giải Thích Codebase Chi Tiết

Dự án: OpcUaCommunicationEngine (AVS / Junction brand)

Stack: WPF .NET 8, C#, OPC UA SDK 1.5.374, Serilog, BCrypt.Net, Newtonsoft.Json, ASP.NET Core, Microsoft.Extensions.DependencyInjection

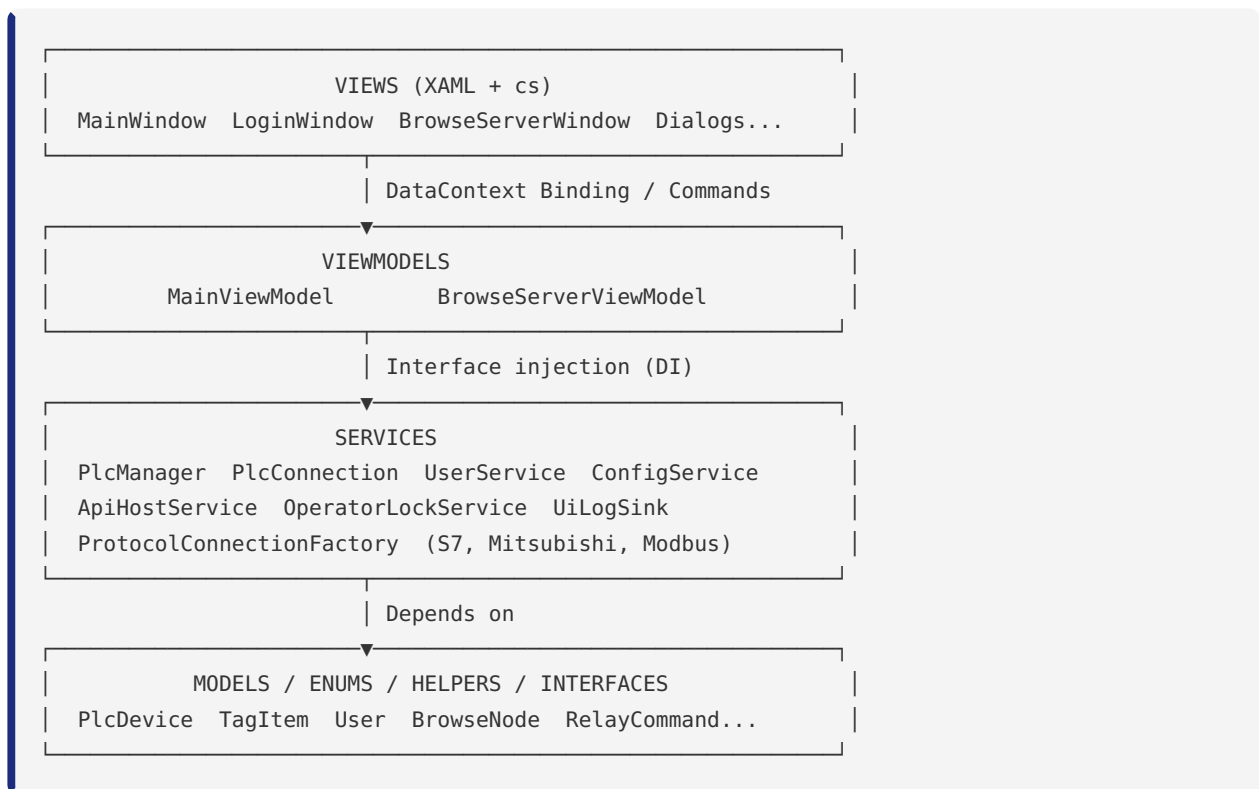
Mục đích: Desktop app kết nối PLC qua OPC UA, duyệt node tree, quản lý tag/subscription, REST API, phân quyền user, operator locking.

Ngày tạo tài liệu: 2026-06-19

MỤC LỤC

1. Tổng quan kiến trúc
 2. Thứ tự build lại từ đầu
 3. Layer 1 — Enums
 4. Layer 2 — Models
 5. Layer 3 — Helpers (RelayCommand, Converters)
 6. Layer 4 — Interfaces
 7. Layer 5 — Services (Auth, Config, OpcUa, Protocols)
 8. Layer 6 — ViewModels
 9. Layer 7 — App startup (App.xaml.cs)
 10. Layer 8 — Views (XAML + code-behind)
 11. Các bug đã fix trong dự án
 12. Patterns và quyết định kiến trúc
-

1. TỔNG QUAN KIẾN TRÚC



Tại sao dùng MVVM? WPF được thiết kế tối ưu cho MVVM. XAML binding trực tiếp vào ViewModel property — khi property thay đổi, UI tự cập nhật mà không cần code-behind. Điều này giúp tách biệt logic (ViewModel) khỏi giao diện (View), dễ test và maintain.

Tại sao dùng Dependency Injection? Khi class A cần class B, thay vì `new B()` bên trong A (tight coupling), ta inject B qua constructor. Lợi ích: (1) dễ thay thế implementation — đổi `PlcManager` thành mock khi test, (2) vòng đời được quản lý tập trung trong DI container, (3) tránh singleton tự quản lý (anti-pattern).

2. THỨ TỰ BUILD LẠI TỪ ĐẦU

Khi rebuild từ đầu, luôn build theo thứ tự phụ thuộc — file nào không phụ thuộc gì thì build trước:

Bước 1: Enums/	← không phụ thuộc gì
Bước 2: Models/ (ObservableObject)	← chỉ cần System + WPF
Bước 3: Models/ (các model khác)	← phụ thuộc Enums + ObservableObject
Bước 4: Interfaces/	← phụ thuộc Models
Bước 5: Helpers/	← phụ thuộc System.Windows.Input
Bước 6: Services/Auth/UserService	← BCrypt.Net + Models
Bước 7: Services/ConfigService	← Newtonsoft.Json + Models
Bước 8: Services/UiLogSink	← Serilog + Models
Bước 9: Services/OpcUa/PlcConn	← OPC UA SDK + tất cả trên
Bước 10: Services/OpcUa/PlcManager	← PlcConnection + Interfaces
Bước 11: Services/Protocols/	← Protocol libs (S7Net, EasyModbus)
Bước 12: Api/	← ASP.NET Core + PlcManager
Bước 13: ViewModels/	← Services + Helpers
Bước 14: App.xaml + App.xaml.cs	← DI wiring + startup
Bước 15: Views/ (Dialogs)	← ViewModels + XAML
Bước 16: Views/ (MainWindow)	← Tất cả trên

Quy tắc: **không bao giờ** để tầng dưới import tầng trên. Models không import ViewModel. Services không import Views.

3. LAYER 1 — ENUMS

Enums là những kiểu dữ liệu liệt kê — không có logic, không có dependencies. Nên viết trước tiên.

UserRole

```
public enum UserRole
{
    Viewer    = 0,    // chỉ xem, không ghi
    Operator  = 1,    // có thể ghi tag (cần có operator lock)
    Admin     = 2     // toàn quyền, bao gồm quản lý user
}
```

Tại sao có giá trị số? Để so sánh phân quyền: `if (role >= UserRole.Operator)`. Không cần switch/if phức tạp. Giá trị 0/1/2 lưu vào JSON và database tốt hơn string.

PlcConnectionState

```
public enum PlcConnectionState
{
    Disabled,          // PLC bị vô hiệu hóa trong config (IsEnabled=false)
    Connecting,        // đang thực hiện kết nối
    Connected,         // kết nối thành công, session active
    Disconnecting,     // đang ngắt kết nối (có thể mất thời gian)
    Disconnected,      // đã ngắt kết nối sạch sẽ
    Reconnecting,      // mất kết nối, đang thử lại
    Error              // lỗi không phục hồi được
}
```

Tại sao cần nhiều state? Để UI hiển thị đúng: nút "Kết nối" disabled khi Connecting/ Reconnecting. Màu icon khác nhau cho từng state. **Disconnecting** riêng biệt với **Disconnected** vì có thể mất 1-5 giây để close session sạch.

OpcUaSecurityPolicy

```
public enum OpcUaSecurityPolicy
{
    None,              // không mã hóa – dùng cho lab/test
    Basic128Rsa15,     // mã hóa AES-128 – cũ, ít dùng
    Basic256,          // mã hóa AES-256
    Basic256Sha256,    // mã hóa AES-256 + SHA-256 – phổ biến nhất
    Aes128Sha256RsaOaep, // mới hơn, nhanh hơn
    Aes256Sha256RsaPss  // mạnh nhất
}
```

Lưu ý thực tế: Siemens S7-1200 mất 6-14 giây để renew Secure Channel với **Basic256Sha256**. Điều này gây timeout và reconnect không cần thiết. Khuyến nghị dùng **None** cho môi trường nội bộ.

TagQuality

```
public enum TagQuality { Unknown, Good, Bad, Uncertain }
```

Map trực tiếp từ OPC UA **StatusCode**. **Unknown** dùng cho lúc chưa đọc lần nào. Hiển thị màu khác nhau trên UI.

4. LAYER 2 — MODELS

Models/ObservableObject.cs — Nền tảng WPF Binding

```
public abstract class ObservableObject : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;
```

`INotifyPropertyChanged` là interface của WPF/XAML. Khi một property thay đổi, WPF cần được thông báo để re-render binding tương ứng. Nếu không implement interface này, binding sẽ chỉ đọc một lần lúc khởi tạo và không bao giờ cập nhật.

```
protected virtual void OnPropertyChanged([CallerMemberName] string? propertyName = null)
{
    var handler = PropertyChanged;
    if (handler == null) return;
```

`[CallerMemberName]` là C# attribute đặc biệt: khi gọi `OnPropertyChanged()` bên trong property setter mà không truyền tên, compiler tự điền tên property. Ví dụ: gọi trong `set` của property `Name` → `propertyName = "Name"`. Điều này tránh lỗi typo khi viết tên string thủ công.

Lưu `handler` vào local variable trước — pattern thread-safety: tránh race condition khi subscriber unsubscribe đúng lúc ta đang invoke.

```
if (Application.Current?.Dispatcher != null &&
    !Application.Current.Dispatcher.CheckAccess())
{
    Application.Current.Dispatcher.BeginInvoke(() =>
    {
        handler.Invoke(this, new PropertyChangedEventArgs(propertyName));
    });
}
else
{
    handler.Invoke(this, new PropertyChangedEventArgs(propertyName));
}
```

Đây là điểm quan trọng nhất của ObservableObject. WPF chỉ cho phép cập nhật UI từ UI thread (main thread). Nhưng OPC UA subscription callback chạy trên background thread — khi giá trị tag thay đổi, nó gọi `tag.UpdateValue()` → `OnPropertyChanged()` từ background thread.

`Dispatcher.CheckAccess()` trả về `true` nếu đang trên UI thread. Nếu `false` (background thread) → `BeginInvoke` để post action vào UI thread queue, async không chờ. Nếu đang trên UI thread → invoke trực tiếp.

```
protected bool SetProperty<T>(ref T field, T value, [CallerMemberName] string? propertyName = null)
{
    if (EqualityComparer<T>.Default.Equals(field, value))
        return false;    // không thay đổi → không notify

    field = value;
    OnPropertyChanged(propertyName);
    return true;          // đã thay đổi
}
}
```

`SetProperty<T>` là shorthand pattern: kiểm tra thay đổi + set + notify trong một dòng. Tất cả property setter trong Models và ViewModels dùng pattern này:

```
public string Name
{
    get => _name;
    set => SetProperty(ref _name, value); // một dòng thay vì 5 dòng
}
```

`EqualityComparer<T>.Default.Equals()` dùng generic equality — đúng cho cả value type (int, bool) lẫn reference type (string, object). Với string, `"abc".Equals("abc")` trả `true` dù là 2 instance khác nhau.

Models/PlcDevice.cs — Model đại diện cho một PLC

`PlcDevice` kế thừa `ObservableObject` vì nó cần binding trực tiếp lên UI (ListBox, DataGrid). Mỗi PLC hiển thị state, connection status, tags — tất cả đều live-update.

Tại sao dùng private backing field thay vì auto-property?

```
// Auto-property – KHÔNG dùng cho binding
public string Name { get; set; }

// Backing field + SetProperty – dùng cho binding
private string _name = string.Empty;
public string Name
{
    get => _name;
    set => SetProperty(ref _name, value);
}
```

Auto-property không có cách gọi `OnPropertyChanged` — không thể binding two-way. Backing field + `SetProperty` mới có thể notify WPF.

[JsonIgnore] trên runtime state:

```
[JsonIgnore]
public PlcConnectionState ConnectionState
{
    get => _connectionState;
    set => SetProperty(ref _connectionState, value);
}
```

`[JsonIgnore]` của `Newtonsoft.Json` — khi serialize `PlcDevice` ra file JSON (save config), ta không muốn lưu `ConnectionState` vì đó là trạng thái runtime, không phải config. Mỗi lần mở app trạng thái reset về `Disconnected`.

Computed properties với `[JsonIgnore]`:

```
[JsonIgnore]
public string ConnectionStateText => ConnectionState switch
{
    PlcConnectionState.Connected => "Connected",
    PlcConnectionState.Error => $"Error: {LastError}",
    PlcConnectionState.Reconnecting => "Reconnecting...",
    _ => ConnectionState.ToString()
};
```

Property computed (không có setter, tính từ state khác). Khi `ConnectionState` thay đổi, phải manually raise `OnPropertyChanged(nameof(ConnectionStateText))` nếu muốn binding cập nhật. Thường làm trong `ConnectionState` setter:

```
set
{
    if (SetProperty(ref _connectionState, value))
    {
        OnPropertyChanged(nameof(ConnectionStateText));
        OnPropertyChanged(nameof(IsConnected));
    }
}
```

Factory methods:

```
public static PlcDevice Create(string name, string endpointUrl)
{
    return new PlcDevice
    {
        Id = Guid.NewGuid().ToString(),
        Name = name,
        EndpointUrl = endpointUrl,
        ProtocolType = ProtocolType.OpcUa,
        ConnectionState = PlcConnectionState.Disabled
    };
}
```

Static factory thay vì constructor có nhiều tham số — dễ đọc hơn, tên method nói lên ngữ cảnh (`CreateSiemensS7` vs `CreateMitsubishiMc`). `Guid.NewGuid().ToString()` tạo ID unique dạng "550e8400-e29b-41d4-a716-446655440000".

Clone() method:

```
public PlcDevice Clone()
{
    return new PlcDevice
    {
        Id = this.Id,
        Name = this.Name,
        // copy tất cả config fields...
        // KHÔNG copy runtime state (ConnectionState, LastError, ...)
    };
}
```

Dùng trong `EditPlcDialog`: lấy bản copy, user chỉnh sửa copy, nếu cancel thì không ảnh hưởng gốc, nếu save thì merge lại. Tránh mutate trực tiếp trong lúc user đang edit.

Models/TagItem.cs — Model đại diện cho một OPC UA Tag

Tương tự `PlcDevice`, `TagItem` cũng kế thừa `ObservableObject` vì nó binding vào `DataGrid` — giá trị, quality, timestamp live-update khi subscription nhận data mới.

UpdateValue() — cập nhật tất cả fields cùng lúc:

```
public void UpdateValue(object? newValue, TagQuality quality, DateTime timestamp, DateTime? serverTimestamp)
{
    PreviousValue = Value; // lưu giá trị cũ để so sánh
    Value = newValue;      // set mới → triggers DisplayValue, HasValueChanged
    Quality = quality;     // Good/Bad/Uncertain
    Timestamp = timestamp; // client-side timestamp
    ServerTimestamp = serverTimestamp;
    LastError = string.Empty; // xóa lỗi cũ khi có data mới
}
```

`PreviousValue = Value` phải gán trước `Value = newValue` — nếu đổi thứ tự, `PreviousValue` sẽ bằng `Value` mới.

HasValueChanged:

```
[JsonIgnore]
public bool HasValueChanged => !Equals(Value, PreviousValue);
```

Dùng `Equals()` thay vì `==` vì `Value` là `object?` — với reference type, `==` so sánh reference, không so sánh nội dung. `Equals()` gọi virtual method, cho phép type cụ thể (int, double, string) override với so sánh giá trị.

Models/User.cs — Model tài khoản người dùng

```
public class User
{
    public string Id { get; set; } = Guid.NewGuid().ToString();
    public string Username { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public string DisplayName { get; set; } = string.Empty;
    public UserRole Role { get; set; } = UserRole.Viewer;
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? LastLoginAt { get; set; }
    public int? LockDurationMinutes { get; set; }
}
```

User KHÔNG kế thừa ObservableObject — không cần binding trực tiếp.

UserManagementWindow dùng DataGrid với `ItemsSource` refresh toàn bộ danh sách khi có thay đổi.

`PasswordHash` — **KHÔNG BAO GIỜ** lưu password plaintext. BCrypt hash là chuỗi 60 ký tự dạng `$2a$11$...`. Ngay cả khi file JSON bị lộ, attacker không thể reverse-engineer password.

`LockDurationMinutes` là `int?` (nullable) — null nghĩa là dùng default từ `AuthSettings`. 0 nghĩa là không giới hạn thời gian (unlimited lock).

`UserStorage` là wrapper class để serialize/deserialize toàn bộ danh sách users:

```
public class UserStorage
{
    public List<User> Users { get; set; } = new();
}
```

Tại sao cần wrapper thay vì serialize trực tiếp `List<User>`? Để dễ mở rộng sau — có thể thêm metadata (version, createdAt, checksum) vào file mà không phá vỡ format cũ.

5. LAYER 3 — HELPERS

Helpers/RelayCommand.cs — Cầu nối ViewModel → View

Vấn đề MVVM cần giải quyết: XAML Button cần `Command` (ICommand), không phải `Click` event. ViewModel có method `void SaveConfig()` — cần bọc nó thành `ICommand`.

RelayCommand (synchronous):

```
public class RelayCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Func<object?, bool>? _canExecute;

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }
}
```

`CommandManager.RequerySuggested` là WPF mechanism tự động re-evaluate `CanExecute` khi UI state thay đổi (focus change, input change). Bằng cách forward `CanExecuteChanged` sang `RequerySuggested`, mọi lần WPF hỏi "button này có enable không?" đều gọi `CanExecute()` của ta.

```
public RelayCommand(Action execute, Func<bool>? canExecute = null)
    : this(_ => execute(), canExecute != null ? _ => canExecute() : null)
{
}
}
```

Overload thứ hai nhận `Action` (không có parameter) và `Func<bool>` (không có parameter) — dễ dùng hơn khi không cần `CommandParameter`:

```
SaveCommand = new RelayCommand(Save, () => HasUnsavedChanges);
// thay vì
SaveCommand = new RelayCommand(_ => Save(), _ => HasUnsavedChanges);
```

AsyncRelayCommand (asynchronous):

```
public class AsyncRelayCommand : ICommand
{
    private bool _isExecuting;

    public bool CanExecute(object? parameter)
        => !_isExecuting && (_canExecute == null || _canExecute());
}
```

`!_isExecuting` ngăn user click nhiều lần khi command đang chạy. Khi `_isExecuting = true`, `CanExecute()` trả về false → button tự disabled.

```
public async void Execute(object? parameter)
{
    if (!CanExecute(parameter)) return;

    try
    {
        IsExecuting = true;    // disables button
        await _execute();      // chạy async task
    }
    finally
    {
        IsExecuting = false;   // re-enables button dù có exception
    }
}
```

`async void` trên `ICommand.Execute` là **hợp lệ** vì `ICommand.Execute` có signature `void Execute(object)`. Ta không thể return `Task` từ đây. Exception trong `async void` sẽ propagate lên `AppDomain.UnhandledException` — đó là lý do cần global exception handler.

`finally` đảm bảo button luôn được re-enable dù task throw exception — không để UI bị kẹt.

6. LAYER 4 — INTERFACES

Interfaces định nghĩa **contract** (hợp đồng) giữa các layer, không có implementation.

Cho phép:

- Test với mock objects
- Thay đổi implementation mà không ảnh hưởng caller
- DI container inject đúng implementation

IPlcConnection — hợp đồng cho 1 kết nối PLC

```
public interface IPlcConnection : IDisposable
{
    PlcDevice Device { get; }
    PlcConnectionState ConnectionState { get; }
    bool IsConnected { get; }
    string? SessionId { get; }

    event EventHandler<ConnectionStateChangedEventArgs>? ConnectionStateChanged;
    event EventHandler<TagValueChangedEventArgs>? TagValueChanged;
    event EventHandler<PlcErrorEventArgs>? ErrorOccurred;

    Task<bool> ConnectAsync(CancellationTokens cancellationTokens = default);
    Task DisconnectAsync();
    Task<bool> ReconnectAsync(CancellationTokens cancellationTokens = default);
    Task<TagValue?> ReadTagAsync(string nodeId, CancellationTokens cancellationTokens = default);
    Task<bool> WriteTagAsync(string nodeId, object value, CancellationTokens cancellationTokens = default);
    Task<IReadOnlyList<BrowseNode>> BrowseAsync(string? nodeId = null, CancellationTokens cancellationTokens = default);
    Task<NodeInfo?> GetNodeInfoAsync(string nodeId, CancellationTokens cancellationTokens = default);
}
```

IDisposable — kết nối cần cleanup (đóng session, hủy subscription). Khi `PlcManager.Dispose()`, nó gọi `connection.Dispose()` cho tất cả connections.

IReadOnlyList<T> thay vì **List<T>** — caller không thể modify list trả về (immutable interface). Tránh side effect không mong muốn.

IPlcManager — quản lý nhiều PLCs

```
public interface IPlcManager : IDisposable
{
    event EventHandler<ConnectionStateChangedEventArgs>? ConnectionStateChanged;
    event EventHandler<TagValueChangedEventArgs>? TagValueChanged;

    Task<IPlcConnection?> AddPlcAsync(PlcDevice device, ...);
    Task<bool> RemovePlcAsync(string plcId, ...);
    IPlcConnection? GetConnection(string plcId);
    Task<bool> ConnectAsync(string plcId, ...);
    Task DisconnectAsync(string plcId, ...);
    Task<int> ConnectAllAsync(...);
    Task<IReadOnlyList<BrowseNode>> BrowseAsync(string plcId, string? nodeId = null, ...);
    PlcStatus? GetStatus(string plcId);
}
```

MainViewModel inject **IPlcManager** không phải **PlcManager** cụ thể. Trong unit test, có thể inject **MockPlcManager** implement interface này.

7. LAYER 5 — SERVICES

Services/UiLogSink.cs — Đưa log lên UI

Vấn đề: Serilog mặc định ghi log ra file và console. Ta muốn log xuất hiện trực tiếp trong cửa sổ app (log panel).

Giải pháp: Viết custom Serilog sink implement `ILogEventSink`.

```
public class UiLogSink : ILogEventSink
{
    private readonly ObservableCollection<LogEntry> _logEntries;
    private readonly System.Windows.Threading.Dispatcher _dispatcher;
    private readonly int _maxEntries;

    public UiLogSink(ObservableCollection<LogEntry> logEntries, int maxEntries = 1000)
    {
        _logEntries = logEntries;
        _dispatcher = System.Windows.Application.Current.Dispatcher; // capture UI dispatcher
        _maxEntries = maxEntries;
    }
}
```

`Dispatcher` được capture lúc constructor — constructor chạy trên UI thread, nên `Application.Current.Dispatcher` là đúng. Nếu capture sau này (khi Emit được gọi từ background thread), `Application.Current` có thể null.

```
public void Emit(LogEvent logEvent)
{
    var entry = new LogEntry
    {
        Timestamp = logEvent.Timestamp.LocalDateTime, // convert từ DateTimeOffset
        Level = GetLevelShortName(logEvent.Level),     // "INF", "WRN", "ERR", ...
        Message = logEvent.RenderMessage(),            // format message với parameters
        Exception = logEvent.Exception?.ToString()     // full stack trace nếu có
    };

    _dispatcher.InvokeAsync(() =>
    {
        _logEntries.Add(entry);

        while (_logEntries.Count > _maxEntries)
            _logEntries.RemoveAt(0); // xóa entry cũ nhất, giữ max 500 entries
    });
}
```

`logEvent.RenderMessage()` format message template với values. Ví dụ: `"Connected to {PlcName}"` với `PlcName="PLC1"` → `"Connected to PLC1"`.

`while (_logEntries.Count > _maxEntries)` loop thay vì `if` — phòng trường hợp burst nhiều logs cùng lúc.

```
public static class UiLogSinkExtensions
{
    public static LoggerConfiguration UiSink(
        this LoggerSinkConfiguration sinkConfig,
        ObservableCollection<LogEntry> logEntries,
        int maxEntries = 1000)
        => sinkConfig.Sink(new UiLogSink(logEntries, maxEntries));
}
```

Extension method cho phép dùng fluent API của Serilog:

```
Log.Logger = new LoggerConfiguration()
    .WriteTo.File("log.txt")
    .WriteTo.UiSink(_mainViewModel.LogEntries, maxEntries: 500) // extension method
    .CreateLogger();
```

Services/Auth/UserService.cs — Quản lý users

```
public class UserService
{
    private readonly string _usersFilePath;
    private readonly object _lock = new(); // thread-safety lock
    private UserStorage _storage;
    private static ILogger Logger => Log.Logger; // dynamic, not captured
```

`private readonly object _lock = new()` — object lock đơn giản, không dùng static (tránh deadlock với singleton). Tất cả public methods đều wrap trong `lock(_lock)` vì: - UI thread gọi khi admin thêm/sửa user - Background API thread gọi khi xác thực login API

```
public UserService(AuthSettings settings)
{
    _usersFilePath = settings.UsersFilePath;
    _storage = LoadOrCreateStorage(); // load ngay trong constructor
}
```

Load users ngay trong constructor — không lazy-load. DI container tạo UserService là Singleton, nên chỉ load 1 lần lúc startup.

LoadOrCreateStorage() — self-healing:

```

private UserStorage LoadOrCreateStorage()
{
    try
    {
        if (File.Exists(_usersFilePath))
        {
            var json = File.ReadAllText(_usersFilePath);
            var storage = JsonConvert.DeserializeObject<UserStorage>(json);
            if (storage != null && storage.Users.Any())
            {
                Logger.Information("Loaded {Count} users from storage", storage.Users.Count);
                return storage;
            }
        }
    }
    catch (Exception ex)
    {
        Logger.Error(ex, "Error loading users storage, creating default");
    }

    // Tự tạo admin mặc định nếu file không có hoặc corrupt
    var defaultStorage = CreateDefaultStorage();
    SaveStorageInternal(defaultStorage);
    return defaultStorage;
}

```

Tại sao cần self-healing? Lần đầu deploy app, file users.json chưa có. Thay vì crash, app tự tạo `admin/admin123`. Tương tự khi file bị corrupt (disk lỗi) — app phục hồi thay vì không cho ai login được.

HashPassword / VerifyPassword:

```

private static string HashPassword(string password)
{
    return BCrypt.Net.BCrypt.HashPassword(password, workFactor: 11);
}

private static bool VerifyPassword(string password, string hash)
{
    try
    {
        return BCrypt.Net.BCrypt.Verify(password, hash);
    }
    catch
    {
        return false; // hash corrupt → false, không crash
    }
}

```

workFactor: 11 — BCrypt tính toán $2^{11} = 2048$ vòng hash. Khoảng 0.15-0.3 giây mỗi lần hash — đủ để brute force 1 tỉ password mất hàng nghìn năm, nhưng không làm người dùng cảm thấy chậm khi login.

workFactor: 12 (= 4096 vòng) sẽ mất 0.3-0.6 giây — dùng khi security quan trọng hơn UX.

ValidateCredentials — timing attack prevention:

```
public User? ValidateCredentials(string username, string password)
{
    lock (_lock)
    {
        var user = _storage.Users.FirstOrDefault(u =>
            u.Username.Equals(username, StringComparison.OrdinalIgnoreCase) && u.IsActive);

        if (user == null) return null;          // user không tồn tại

        if (VerifyPassword(password, user.PasswordHash))
        {
            user.LastLoginAt = DateTime.UtcNow;
            SaveStorage();
            return user;
        }

        return null;    // sai password — cũng trả null như user không tồn tại
    }
}
```

Cả "user không tồn tại" và "sai password" đều trả về **null** — không lộ thông tin cho attacker biết cái nào sai. (Nếu phân biệt, attacker có thể enumerate username hợp lệ.)

StringComparison.OrdinalIgnoreCase — "Admin" == "admin" == "ADMIN". Username case-insensitive.

DeleteUser — bảo vệ last admin:


```

public bool DeleteUser(string userId)
{
    lock (_lock)
    {
        var user = _storage.Users.FirstOrDefault(u => u.Id == userId);
        if (user == null) return false;

        if (user.Role == UserRole.Admin)
        {
            var adminCount = _storage.Users.Count(u => u.Role == UserRole.Admin && u.IsActive);
            if (adminCount <= 1)
            {
                throw new InvalidOperationException("Cannot delete the last admin user");
            }
        }

        _storage.Users.Remove(user);
        SaveStorage();
        return true;
    }
}

```

Ném exception thay vì return false — caller (EditUserDialog) phải xử lý exception này để hiển thị message cụ thể cho user. Return false chỉ nên dùng khi "not found" — đó là expected case, không phải error.

Services/ConfigurationService.cs — Đọc/ghi cấu hình

```

public class ConfigurationService : IConfigurationService
{
    private readonly ILogger _logger;
    private readonly string _defaultConfigPath;
    private AppConfiguration _currentConfiguration;
    private string? _currentFilePath;
    private bool _hasUnsavedChanges;

    public event EventHandler<AppConfiguration>? ConfigurationChanged;
}

```

ConfigurationChanged event — khi load file mới, MainViewModel cần biết để refresh UI. Event pattern tốt hơn polling.

```

public ConfigurationService(ILogger logger, string? defaultConfigPath = null)
{
    _defaultConfigPath = defaultConfigPath ?? Path.Combine(
        AppDomain.CurrentDomain.BaseDirectory,
        "Configurations",
        "plc_config.json");

    _currentConfiguration = AppConfiguration.CreateDefault();
}

```

`AppDomain.CurrentDomain.BaseDirectory` — thư mục chứa file .exe. Tốt hơn `Environment.CurrentDirectory` vì `CurrentDirectory` có thể thay đổi nếu user chạy từ shortcut hoặc command line ở thư mục khác.

Không load config trong constructor — để cho `InitializeAsync()` trong `MainViewModel` gọi sau khi DI wired xong.

GetJsonSettings():

```

private static JsonSerializerSettings GetJsonSettings()
{
    return new JsonSerializerSettings
    {
        Formatting = Formatting.Indented,           // JSON đọc được bằng mắt
        NullValueHandling = NullValueHandling.Ignore, // không ghi null fields
        DefaultValueHandling = DefaultValueHandling.Include, // ghi default values
        DateFormatString = "yyyy-MM-dd HH:mm:ss" // format dễ đọc, không ISO 8601
    };
}

```

`NullValueHandling.Ignore` — khi serialize, bỏ qua properties null → file JSON gọn hơn. Ví dụ: `"CertificatePath": null` không xuất hiện trong file nếu không dùng.

`Formatting.Indented` — file JSON có indent, dễ đọc và diff trong git. Trade-off: file to hơn nhưng chấp nhận được vì config file không đến vài MB.

Services/OpcUa/PlcConnection.cs — Trái tim OPC UA

Tại sao `private static ILogger Logger => Log.Logger` (property) thay vì `private readonly ILogger _logger` (field)?

```

// Sai — captures logger tại thời điểm construction
private readonly ILogger _logger;
public PlcConnection(PlcDevice device, ILogger logger) { _logger = logger; }

```

DI container tạo PlcConnection trước khi UiSink được add vào Serilog. Nếu capture logger qua field, tất cả logs từ PlcConnection sẽ không xuất hiện trên UI (UiSink chưa được add).

```
// Đúng – lấy logger mới nhất mỗi lần gọi
private static ILogger Logger => Log.Logger;
```

`Log.Logger` là static property của Serilog — luôn trả về instance hiện tại. Sau khi App.xaml.cs tạo lại logger với UiSink, mọi `Logger.Information(...)` từ PlcConnection sẽ tự động đến UI.

Fields phức tạp — giải thích từng cái:

```
private readonly object _lock = new();
```

Object lock cho các thao tác cần atomic trên `_reconnectHandler` và `_isReconnecting`. `lock(obj)` đảm bảo chỉ một thread chạy trong block tại một thời điểm.

```
private bool _isDisconnecting;
```

Flag ngăn KeepAlive event trigger reconnect trong khi đang disconnect. Nếu không có flag này: `DisconnectAsync()` đang chạy → KeepAlive fires vì session đang đóng → trigger reconnect → reconnect thất bại → vòng lặp vô tận.

```
private bool _isReconnecting;
```

Flag ngăn nhiều reconnect loops chạy song song. Không có flag: lần kết nối thất bại 1 → start AutoReconnect loop 1. Loop 1 thất bại lần đầu → start AutoReconnect loop 2. Tiếp tục nhân đôi → resource leak.

```
private SessionReconnectHandler? _reconnectHandler;
```

Class của OPC UA SDK. Xử lý 2-layer reconnection: (1) Secure Channel (TLS) renewal; (2) Session resumption. Phức tạp hơn đơn giản là `ConnectAsync()` lại vì phải giữ subscription state.

```
private DateTime _lastReconnectCompleteTime = DateTime.MinValue;
private const int KeepAliveSettlingPeriodMs = 2000;
```

Sau khi reconnect xong, OPC UA session có thể gửi vài KeepAlive "stale" từ kết nối cũ. Nếu không ignore, chúng kích hoạt reconnect lần 2 ngay lập tức — race condition. Settling period 2 giây bỏ qua các KeepAlive này.

```
private readonly ConcurrentDictionary<string, Subscription> _subscriptions = new();
private readonly ConcurrentDictionary<uint, (string TagId, string NodeId)> _monitoredItemMapping = new();
```

`ConcurrentDictionary` vì subscriptions được tạo/xóa từ cả UI thread (khi user add/remove tag) lẫn reconnect thread.

`_monitoredItemMapping` map `ClientHandle` (uint, OPC UA internal ID) → `(TagId, NodeId)`.
Khi `MonitoredItem_Notification` callback fires, ta lookup tag cần update.

ConnectAsync() — chuỗi kết nối:

```
public async Task<bool> ConnectAsync(CancellationTokentoken = default)
{
    if (_disposed) throw new ObjectDisposedException(nameof(PlcConnection));
    if (IsConnected) { Logger.Warning("Already connected"); return true; }

    try
    {
        ConnectionState = PlcConnectionState.Connecting;
        // Bước 1: tạo OPC UA application config
        _appConfig = await CreateApplicationConfigurationAsync();
        // Bước 2: discover endpoints và chọn tốt nhất
        var selectedEndpoint = await SelectEndpointAsync(_device.EndpointUrl, cancellationTokentoken);
        // Bước 3: tạo session
        _session = await Session.Create(_appConfig, endpoint, false, appName, sessionTimeout, userIdentity,
        // Bước 4: đăng ký events
        _session.KeepAlive += Session_KeepAlive;
        _session.Notification += Session_Notification;
        _session.PublishError += Session_PublishError;
        // Bước 5: update state
        ConnectionState = PlcConnectionState.Connected;
        LastConnectedTime = DateTime.Now;
        LastError = null;
        // Bước 6: tạo subscriptions
        foreach (var group in _device.SubscriptionGroups.Where(g => g.IsEnabled))
            await CreateSubscriptionAsync(group, cancellationTokentoken);
        return true;
    }
    catch (Exception ex)
    {
        LastError = ex.Message;
        ConnectionState = PlcConnectionState.Error;
        // Log chi tiết OPC UA status code nếu có
        if (ex is ServiceResultException sre)
            Logger.Error("OPC UA StatusCode: 0x{Code:X8}", sre.StatusCode);
        // Bắt đầu auto-reconnect nếu được cấu hình
        if (_device.AutoReconnect && !_isReconnecting && !_isDisconnecting)
            StartAutoReconnect();
        return false;
    }
}
```

`ServiceResultException` là exception type của OPC UA SDK — chứa `StatusCode` dạng hex (0x80350000 = BadNotConnected, ...). Log hex code giúp debug chính xác.

CreateApplicationConfigurationAsync() — certificate và timeouts:

```
TransportQuotas = new TransportQuotas
{
    OperationTimeout = 60000,           // 60 giây – vì S7-1200 mất 6-14s renew Secure Channel
    SecurityTokenLifetime = 3600000     // 1 giờ – tần suất renew Secure Channel
},
```

S7-1200 với `Basic256Sha256` mất 6-14 giây để renew Secure Channel. Nếu `OperationTimeout = 15000ms` (default), timeout xảy ra trong lúc renew → KeepAlive fail → reconnect → lại renew → vòng lặp. Tăng lên 60s giải quyết vấn đề này.

```
config.CertificateValidator = new CertificateValidator();
config.CertificateValidator.CertificateValidation += (validator, e) =>
{
    e.Accept = true; // chấp nhận mọi certificate – chỉ dùng cho lab/internal
};
```

Trong môi trường factory/nội bộ, PLC thường dùng self-signed certificate. Nếu validate strict, kết nối sẽ bị từ chối. `e.Accept = true` bỏ qua kiểm tra certificate — **KHÔNG dùng trong môi trường internet-facing**.

SelectBestEndpoint() — ưu tiên No Security:

```
private EndpointDescription SelectBestEndpoint(EndpointDescriptionCollection endpoints)
{
    // Priority 1: None security (khuyến nghị cho S7-1200 stability)
    if (!useSecurity)
    {
        var noneEndpoint = endpoints
            .Where(e => e.SecurityMode == MessageSecurityMode.None)
            .OrderBy(e => e.SecurityLevel)
            .FirstOrDefault();
        if (noneEndpoint != null) return noneEndpoint;
    }
    // Priority 2: Exact match
    // Priority 3: Policy match
    // Priority 4: Lighter security (Basic128 hoặc Basic256 không Sha256)
    // Priority 5: Any secure endpoint
    // Last: first available
}
```

Logic này đặc thù cho Siemens S7-1200/1500. PLC expose nhiều endpoints với security khác nhau. Thứ tự ưu tiên được tối ưu cho stability hơn security — phù hợp factory environment.

Session_KeepAlive() — xử lý mất kết nối:

```

private void Session_KeepAlive(ISession session, KeepAliveEventArgs e)
{
    if (_isDisconnecting || _disposed) return; // ignore khi đang shutdown

    if (e.Status != null && ServiceResult.IsNotGood(e.Status))
    {
        if (_reconnectHandler != null) {
            Logger.Debug("KeepAlive during reconnection"); // handler đã chạy rồi
            return;
        }

        // Kiểm tra settling period – bỏ qua KeepAlive stale sau khi reconnect
        var timeSince = (DateTime.Now - _lastReconnectCompleteTime).TotalMilliseconds;
        if (timeSince < KeepAliveSettlingPeriodMs) {
            Logger.Debug("Ignoring KeepAlive during settling period");
            return;
        }

        if (ConnectionState == PlcConnectionState.Connected && _device.AutoReconnect && !_isReconnecting)
        {
            ConnectionState = PlcConnectionState.Reconnecting;

            lock (_lock)
            {
                if (_reconnectHandler == null) // double-check trong lock
                {
                    _isReconnecting = true;
                    _reconnectHandler = new SessionReconnectHandler(true);
                    _reconnectHandler.BeginReconnect(_session, 10000, SessionReconnectHandler_Complete);
                }
            }
        }
    }
    else
    {
        // KeepAlive thành công – session sống
        if (ConnectionState == PlcConnectionState.Reconnecting)
        {
            // Session tự recover trong lúc handler đang chạy
            lock (_lock)
            {
                _reconnectHandler?.Dispose();
                _reconnectHandler = null;
                _isReconnecting = false;
                _lastReconnectCompleteTime = DateTime.Now;
            }
            ConnectionState = PlcConnectionState.Connected;
        }
    }
}

```

`lock(_lock)` xung quanh `_reconnectHandler = new SessionReconnectHandler()` — double-check locking pattern. KeepAlive có thể fire từ nhiều thread (OPC UA SDK có internal

thread pool). Mà không có lock, 2 threads có thể cùng check `_reconnectHandler == null` trước khi một trong chúng set nó → 2 handlers chạy song song.

BrowseAsync() — lỗi NodeId quan trọng:

```
var startNode = string.IsNullOrEmpty(nodeId)
    ? ObjectIds.ObjectsFolder          // = NodeId với identifier=85
    : NodeId.Parse(nodeId);           // ĐÚNG: Parse "i=85" → Numeric NodeId
                                     // SAI: new NodeId("i=85") → String NodeId
```

`NodeId.Parse("i=85")` → `NodeId` với `NamespaceIndex=0`, `IdentifierType=Numeric`, `Identifier=85`

`new NodeId("i=85")` → `NodeId` với `NamespaceIndex=0`, `IdentifierType=String`, `Identifier="i=85"`

OPC UA server trả về lỗi `BadNodeIdUnknown` khi dùng String NodeId cho node numeric. Bug này không crash — chỉ trả về empty list — gây ra "browse không ra gì" mà khó debug.

Services/OpcUa/PlcManager.cs — Quản lý nhiều connections

```
public class PlcManager : IPlcManager
{
    private readonly ConcurrentDictionary<string, IPlcConnection> _connections = new();
```

`ConcurrentDictionary` vì: - UI thread: Add/Remove khi user thêm/xóa PLC - Background threads: `PlcConnection` sự kiện `ConnectionStateChanged` access dictionary - Auto-reconnect: các connection objects được update từ thread riêng

Factory pattern qua `IProtocolConnectionFactory`:

```
public async Task<IPlcConnection?> AddPlcAsync(PlcDevice device, ...)
{
    if (!_connectionFactory.IsProtocolSupported(device))
    {
        Logger.Error("Protocol {Protocol} not supported", device.ProtocolType);
        return null;
    }

    var connection = _connectionFactory.CreateConnection(device);
```

`ProtocolConnectionFactory.CreateConnection()` nhìn vào `device.ProtocolType` và trả về: - `PlcConnection` cho OpcUa - `SiemensS7Connection` cho SiemensS7 - `MitsubishiMcConnection` cho MitsubishiMc - `ModbusTcpConnection` cho ModbusTcp

`PlcManager` không biết và không cần biết implementation cụ thể — chỉ làm việc qua `IPlcConnection` interface. Đây là Open/Closed Principle: thêm protocol mới chỉ cần thêm class mới trong Factory, không cần sửa `PlcManager`.

Event aggregation pattern:

```
connection.ConnectionStateChanged += OnConnectionStateChanged;  
connection.TagValueChanged += OnTagValueChanged;  
connection.ErrorOccurred += OnErrorOccurred;
```

PlcManager subscribe vào events của mỗi connection. Khi event fires từ bất kỳ connection nào, PlcManager forward lên MainViewModel. MainViewModel chỉ cần subscribe vào PlcManager một chỗ thay vì phải subscribe vào từng connection riêng (và manage lifecycle).

```
private void OnConnectionStateChanged(object? sender, ConnectionStateChangedEventArgs e)  
{  
    Logger.Information("PLC {Name}: {Old} -> {New}", e.PlcName, e.OldState, e.NewState);  
    ConnectionStateChanged?.Invoke(this, e); // forward nguyên event args  
}
```

Log thêm ở đây — tất cả state changes đều được ghi, ngay cả khi MainViewModel không handle.

Services/Auth/OperatorLockService.cs — Hệ thống khoá operator

Use case: Trong môi trường nhiều người vận hành, chỉ 1 người được phép ghi giá trị vào PLC tại một thời điểm để tránh xung đột. OperatorLock giải quyết vấn đề này.

Lock state persistence — tại sao lưu ra file?

```
private readonly string _lockStatePath = "Configurations/lock_state.json";
```

Nếu chỉ lưu trong memory, khi app restart lock biến mất. Khi nhiều instance app chạy (supervisor + operator), chúng cần share lock state. File JSON được chia sẻ qua file system.

FileSystemWatcher — detect external changes:

```
private void StartFileWatcher()  
{  
    _fileWatcher = new FileSystemWatcher(directory, "lock_state.json")  
    {  
        NotifyFilter = NotifyFilters.LastWrite | NotifyFilters.CreationTime | NotifyFilters.FileName,  
        EnableRaisingEvents = true  
    };  
  
    _fileWatcher.Changed += OnLockFileChanged;  
    _fileWatcher.Created += OnLockFileChanged;  
    _fileWatcher.Deleted += OnLockFileDeleted;  
}
```


`FileSystemWatcher` monitor file thay đổi từ bên ngoài — khi app B acquire lock, app A nhận event và cập nhật UI.

Debounce trong file watcher:

```
private void OnLockFileChanged(object sender, FileSystemEventArgs e)
{
    if (_isInternalUpdate) return; // ignore changes ta tự tạo

    var now = DateTime.UtcNow;
    if ((now - _lastFileChange).TotalMilliseconds < 500) return; // debounce 500ms
    _lastFileChange = now;

    Task.Delay(100).ContinueWith(_ => ReloadLockStateFromFile()); // 100ms delay
}
```

`FileSystemWatcher` thường fire nhiều events cho một lần write (Created, Changed, Changed lần nữa). Debounce 500ms gộp chúng thành một xử lý.

`_isInternalUpdate` flag — khi ta tự write file (SaveLockState), không muốn trigger lại watcher của chính mình.

Atomic file write:

```
private void SaveLockState()
{
    _isInternalUpdate = true;
    var tempPath = _lockStatePath + ".tmp";
    var json = JsonConvert.SerializeObject(_currentLock, Formatting.Indented);
    File.WriteAllText(tempPath, json); // write vào temp trước

    if (File.Exists(_lockStatePath)) File.Delete(_lockStatePath);
    File.Move(tempPath, _lockStatePath); // rename atomic (single operation)

    Task.Delay(200).ContinueWith(_ => _isInternalUpdate = false); // reset sau 200ms
}
```

Write trực tiếp vào file có thể để lại file incomplete nếu app crash giữa chừng. Write vào `.tmp` trước, rename sau — rename thường là atomic operation trên hệ điều hành (single OS call). App khác đọc file sẽ thấy hoặc version cũ hoặc version mới, không bao giờ thấy version partial.

TryAcquireLock — logic phức tạp:

```

public (bool success, OperatorLock? operatorLock, string? error, ...) TryAcquireLock(...)
{
    lock (_lock)
    {
        if (role < UserRole.Operator)
            return (false, null, "Insufficient permissions", null, null);

        if (_currentLock != null && !_currentLock.IsExpired)
        {
            if (_currentLock.UserId == userId)
                return ExtendLockInternal(userId, durationMinutes ?? default); // refresh

            return (false, null, "Lock held by another", _currentLock.Username, ...);
        }

        // Calculate expiry
        DateTime expiresAt = (role == UserRole.Admin || durationMinutes == 0)
            ? DateTime.MaxValue // unlimited
            : DateTime.UtcNow.AddMinutes(Math.Min(durationMinutes ?? default, maxDuration));

        _currentLock = new OperatorLock { ... ExpiresAt = expiresAt ... };
        SaveLockState();
        LockAcquired?.Invoke(this, new LockEventArgs { Lock = _currentLock, Reason = "acquired" });
        return (true, _currentLock, null, null, null);
    }
}

```

Tuple return type `(bool success, OperatorLock?, string? error, string? holderUsername, string? holderDisplayName)` — trả về nhiều thông tin trong một call mà không cần class wrapper riêng. Deconstruct ở caller: `var (ok, lockObj, err, holderName, _) = lockService.TryAcquireLock(...)`.

8. LAYER 6 — VIEWMODELS

ViewModels/BrowseServerViewModel.cs

```
public class BrowseServerViewModel : ObservableObject
{
    private readonly PlcConnection _connection;
    private readonly PlcDevice _device;

    public BrowseServerViewModel(PlcConnection connection)
    {
        _connection = connection ?? throw new ArgumentNullException(nameof(connection));
        _device = connection.Device; // convenience reference

        // Initialize commands
        RefreshCommand = new AsyncRelayCommand(RefreshAsync);
        AddSelectedTagCommand = new RelayCommand(AddSelectedTag, () => CanAddSelectedTag);
        // ...

        _ = LoadRootNodesAsync(); // fire and forget – bắt đầu load ngay
    }
}
```

`_ = LoadRootNodesAsync()` trong constructor — dấu `_` discard kết quả Task. Không `await` vì constructor không async. Task chạy ngầm; khi hoàn thành, `RootNodes` được populate và UI tự cập nhật qua binding.

LoadRootNodesAsync() — load OPC UA root folders:

```

private async Task LoadRootNodesAsync()
{
    IsBusy = true;

    var rootFolders = new[]
    {
        ("i=85", "Objects"), // Objects folder – chứa device data
        ("i=86", "Types"),   // Types – kiểu dữ liệu OPC UA
        ("i=87", "Views")    // Views – tùy chỉnh UI của server
    };

    foreach (var (nodeIdStr, fallbackName) in rootFolders)
    {
        var info = await _connection.GetNodeInfoAsync(nodeIdStr);
        var folderNode = new BrowseNodeItem
        {
            NodeId = nodeIdStr,
            DisplayName = info?.DisplayName ?? fallbackName, // fallback nếu null
            HasChildren = true
        };
        folderNode.OnExpandRequested += OnNodeExpandRequested; // lazy load khi expand
        folderNode.AddLoadingPlaceholder(); // thêm "Loading..." item vào tree

        if (nodeIdStr == "i=85")
        {
            await LoadChildrenAsync(folderNode); // eager load Objects
            folderNode.IsExpanded = true;
        }

        RootNodes.Add(folderNode);
    }
}

```

`i=85`, `i=86`, `i=87` là NodeIds chuẩn của OPC UA — mọi OPC UA server đều có chúng. `i=` prefix nghĩa là `IdentifierType=Numeric`.

Chỉ eager-load `i=85` (Objects) vì đó là nơi chứa data PLC — người dùng quan tâm nhất. `Types` và `Views` load on-demand khi expand.

Lazy loading qua event:

```

private async void OnNodeExpandRequested(BrowseNodeItem node)
{
    if (node.ChildrenLoaded) return; // đã load rồi thì thôi
    await LoadChildrenAsync(node);
}

```

`async void` — event handler không thể return Task. Exception trong `async void` propagates lên App global handler.

`ChildrenLoaded` flag tránh load lại nhiều lần khi user expand/collapse nhiều lần.

AddSelectedTag() — thêm tag vào danh sách:

```
private void AddSelectedTag()
{
    if (SelectedNode == null || !SelectedNode.CanAddAsTag) return;

    var subscriptionGroup = _device.SubscriptionGroups.FirstOrDefault(g => g.IsEnabled);
    if (subscriptionGroup == null)
    {
        StatusMessage = "Không có subscription group nào khả dụng";
        return;
    }

    if (_device.Tags.Any(t => t.NodeId == SelectedNode.NodeId))
    {
        StatusMessage = $"Tag '{SelectedNode.DisplayName}' đã tồn tại";
        return;
    }

    var tag = SelectedNode.ToTagItem(_device.Id, subscriptionGroup.Id);
    _device.Tags.Add(tag);          // thêm vào PlcDevice model
    SelectedTags.Add(tag);          // thêm vào collection hiển thị trong window

    StatusMessage = $"Đã thêm tag: {tag.Name}";
}
```

`_device.Tags.Add(tag)` — add trực tiếp vào `PlcDevice.Tags` (ObservableCollection). Sau khi BrowseServerWindow đóng, MainViewModel có thể truy cập `browseWindow.AddedTags` để biết những tag nào được thêm trong session này.

`SelectedTags` là collection riêng chỉ chứa tags được thêm trong session browse hiện tại — dùng để hiển thị trong panel "Tags đã thêm" và để `BrowseServerWindow.AddedTags` property.

ViewModels/MainViewModel.cs

Constructor — wiring commands:

```

public MainViewModel(IConfigurationService configService, IDataCache dataCache,
    IPlcManager plcManager, UserService userService, ILogger logger)
{
    // Store all dependencies
    _configService = configService;
    _plcManager = plcManager;
    // ...

    // Subscribe to PlcManager events
    _plcManager.ConnectionStateChanged += OnPlcConnectionStateChanged;
    _plcManager.TagValueChanged += OnTagValueChanged;
    _plcManager.ErrorOccurred += OnPlcErrorOccurred;

    // LogCount phải update khi collection thay đổi
    LogEntries.CollectionChanged += (_, _) => OnPropertyChanged(nameof(LogCount));

    // Wire all commands
    AddPlcCommand = new RelayCommand(AddPlc);
    ConnectSelectedCommand = new AsyncRelayCommand(ConnectSelectedAsync, () => HasSelectedPlc);
    SaveConfigCommand = new AsyncRelayCommand(SaveConfigurationAsync, () => HasUnsavedChanges);
    BrowseServerCommand = new AsyncRelayCommand(BrowseServerAsync, () => HasSelectedPlc);
    // ...
}

```

Tại sao `LogCount` cần `CollectionChanged` ?

`LogCount` là property computed: `public int LogCount => LogEntries.Count`. Khi `LogEntries.Add()` được gọi, `ObservableCollection` fire `CollectionChanged` — nhưng WPF binding chỉ update controls binding vào `LogEntries` trực tiếp (ListView, ItemsControl), không update `LogCount` vì WPF không biết `LogCount` phụ thuộc vào `LogEntries.Count`. Ta phải manually raise `PropertyChanged` cho `LogCount` mỗi khi collection thay đổi.

`OnPlcConnectionStateChanged` — thread marshalling:

```

private void OnPlcConnectionStateChanged(object? sender, ConnectionStateChangedEventArgs e)
{
    System.Windows.Application.Current?.Dispatcher.InvokeAsync(() =>
    {
        ConnectedPlcCount = _plcManager.ConnectedCount;
        IsConnected = ConnectedPlcCount > 0;
        OnPropertyChanged(nameof(ConnectionStatusText));
        StatusMessage = $"{e.PlcName}: {e.NewState}";

        var plc = PlcDevices.FirstOrDefault(p => p.Id == e.PlcId);
        if (plc != null) plc.ConnectionState = e.NewState;
    });
}

```

Event này đến từ OPC UA session thread — bắt buộc dispatch về UI thread.

`Dispatcher.InvokeAsync` (async, non-blocking) thay vì `Dispatcher.Invoke` (sync, blocking). Blocking có thể gây deadlock nếu UI thread đang chờ background thread.

AddPlc() — luồng đầy đủ:

```
private void AddPlc()
{
    var dialog = new Views.AddPlcDialog { Owner = Application.Current.MainWindow };
    if (dialog.ShowDialog() != true || dialog.Result == null) return;

    var newPlc = dialog.Result;

    // 1. Thêm vào UI list ngay (optimistic update)
    PlcDevices.Add(newPlc);
    _configService.CurrentConfiguration.PlcDevices.Add(newPlc);
    _configService.MarkAsModified();

    // 2. Tạo PlcConnection object (không connect)
    _ = _plcManager.AddPlcAsync(newPlc);

    // 3. Auto-connect nếu user chọn
    if (dialog.ConnectAfterAdd)
    {
        _ = Task.Run(async () =>
        {
            try
            {
                var ok = await _plcManager.ConnectAsync(newPlc.Id);
                Application.Current?.Dispatcher.InvokeAsync(() =>
                {
                    if (ok)
                    {
                        StatusMessage = $"Connected to {newPlc.Name}. Đang mở Browse Server...";
                        OpenBrowseServerForNewPlc(newPlc);
                    }
                    else
                    {
                        StatusMessage = $"Failed to connect to {newPlc.Name}";
                    }
                });
            }
            catch (Exception ex)
            {
                _logger.Error(ex, "Connection error for {Name}", newPlc.Name);
                Application.Current?.Dispatcher.InvokeAsync(() =>
                {
                    StatusMessage = $"Connection error: {ex.Message}";
                });
            }
        });
    }
}
```

`Task.Run()` — đẩy `ConnectAsync` ra background thread vì nó network I/O, có thể mất vài giây. Nếu chạy trên UI thread, window sẽ freeze.

Sau khi connect xong, `Dispatcher.InvokeAsync` trở về UI thread để mở `BrowseServerWindow` (`ShowDialog` phải chạy trên UI thread).

OpenBrowseServerForNewPlc() — type check:

```
private void OpenBrowseServerForNewPlc(PlcDevice plc)
{
    var connection = _plcManager.GetConnection(plc.Id);
    if (connection is not Services.OpcUa.PlcConnection plcConnection)
    {
        // Không phải OPC UA — Siemens S7 / Modbus không có Browse Server
        StatusMessage = $"Connected to {plc.Name}";
        return;
    }

    SelectedPlc = plc;
    var browseWindow = new Views.BrowseServerWindow(plcConnection)
    {
        Owner = Application.Current.MainWindow
    };

    var result = browseWindow.ShowDialog();

    if (result == true && browseWindow.AddedTags.Any())
    {
        OnPropertyChanged(nameof(SelectedPlcTags));
        OnPropertyChanged(nameof(TotalTagCount));
        StatusMessage = $"Đã thêm {browseWindow.AddedTags.Count} tags từ {plc.Name}";
    }
}
```

`connection is not Services.OpcUa.PlcConnection plcConnection` — C# pattern matching.

Nếu `connection` không phải `PlcConnection`, trả về sớm. Nếu là, `plcConnection` variable được set. Chỉ OPC UA có Browse Server.

9. LAYER 7 — APP STARTUP (App.xaml.cs)

OnStartup() — trình tự khởi động

```
protected override void OnStartup(StartupEventArgs e)
{
    base.OnStartup(e);

    // BƯỚC 1: Ngăn app tắt khi LoginWindow đóng
    ShutdownMode = ShutdownMode.OnExplicitShutdown;
```

WPF mặc định: khi cửa sổ đầu tiên đóng → app tắt. Nhưng LoginWindow đóng sau khi login → MainWindow mở. Nếu dùng default, app sẽ tắt ngay khi LoginWindow đóng.

`OnExplicitShutdown` cho phép ta kiểm soát hoàn toàn.


```
// BƯỚC 2: Serilog lần đầu (chưa có UI sink)
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Debug()
    .WriteTo.Console()
    .WriteTo.File("Logs/app-.log", rollingInterval: RollingInterval.Day, retainedFileCountLimit: 7)
    .CreateLogger();
```

Chưa thêm UiSink vì `_mainViewModel.LogEntries` chưa tồn tại. `RollingInterval.Day` = mỗi ngày một file mới. `retainedFileCountLimit: 7` = giữ 7 ngày rồi xóa — không để disk đầy.

```
// BƯỚC 3: Ba global exception handlers
DispatcherUnhandledException += App_DispatcherUnhandledException;
AppDomain.CurrentDomain.UnhandledException += AppDomain_UnhandledException;
TaskScheduler.UnobservedTaskException += TaskScheduler_UnobservedTaskException;
```

Handler	Thread	Có thể prevent crash?	Khi nào fire?
Dispatcher	UI thread	Có (e.Handled=true)	Exception không được catch trong event handler, command
AppDomain	Any	Không (e.IsTerminating)	Exception trong background thread
TaskScheduler	Any	Có (e.SetObserved())	Task throw exception không được awaited

```
// BƯỚC 4: DI setup
var services = new ServiceCollection();
ConfigureServices(services);
_serviceProvider = services.BuildServiceProvider();

// BƯỚC 5: Login
var userService = _serviceProvider.GetRequiredService<UserService>();
var loginWindow = new LoginWindow(userService);
var loginResult = loginWindow.ShowDialog(); // blocks cho đến khi window đóng

if (loginResult != true || loginWindow.LoggedInUser == null)
{
    Shutdown(0); // user cancel login → tắt app
    return;
}

_loggedInUser = loginWindow.LoggedInUser;
```

`ShowDialog()` trả về `bool?`. `true` = DialogResult=true (login success). `false` = DialogResult=false. `null` = window đóng bằng X hoặc Alt+F4.

```
// BƯỚC 6: Tạo MainWindow và MainViewModel từ DI
var mainWindow = _serviceProvider.GetRequiredService<MainWindow>();
_mainViewModel = _serviceProvider.GetRequiredService<MainViewModel>();
_mainViewModel.SetLoggedInUser(_loggedInUser);

// BƯỚC 7: Thêm UiSink vào Serilog
Log.Logger = new LoggerConfiguration()
    // ... same as before ...
    .WriteTo.UiSink(_mainViewModel.LogEntries, maxEntries: 500) // NEW
    .CreateLogger();
```

Serilog được recreate hoàn toàn — không phải add sink vào instance cũ. Đây là thiết kế của Serilog: immutable logger, phải recreate khi muốn thêm sink.

Từ đây trở đi, mọi `Log.Information(...)` trong toàn bộ app (kể cả `PlcConnection`, `PlcManager`, ...) sẽ xuất hiện trên UI panel — vì chúng dùng `Log.Logger` static.

```
// BƯỚC 8: Switch shutdown mode
MainWindow = mainWindow;
ShutdownMode = ShutdownMode.OnMainWindowClose; // bình thường từ đây
mainWindow.Show();

// BƯỚC 9: Initialize async (fire and forget)
_ = InitializeViewModelAsync(_mainViewModel);
}
```

`mainWindow.Show()` trước `_ = InitializeViewModelAsync()` — `MainWindow` hiển thị ngay. Trong lúc load config (có thể mất 1-2 giây), user thấy UI (tuy empty). Nếu `await InitializeViewModelAsync()` trước `Show()`, user thấy màn hình trắng trong 2 giây.

```
private async Task InitializeViewModelAsync(MainViewModel viewModel)
{
    await Task.Delay(100); // đợi 100ms để UI render xong trước
    await viewModel.InitializeAsync();
    await StartApiServerAsync();
}
```

`Task.Delay(100)` — nhỏ nhưng quan trọng. Cho phép WPF message loop xử lý pending render messages. Không có delay, `InitializeAsync` có thể bắt đầu trước khi `MainWindow` fully rendered.

ConfigureServices — chi tiết DI:

```
// Singleton logger – nhưng PlcConnection/PlcManager không dùng cái này
services.AddSingleton<ILogger>(Log.Logger);

// Load ApiSettings từ file trước khi register
_apiSettings = LoadApiSettings();
services.AddSingleton(_apiSettings);

// Transient MainWindow – mỗi lần resolve tạo instance mới
// (trong thực tế chỉ tạo 1 lần)
services.AddTransient<MainWindow>();

// Singleton MainViewModel – tạo 1 lần, sống suốt vòng đời app
services.AddSingleton<MainViewModel>();
```

OnExit() — cleanup có thứ tự:

```
protected override void OnExit(ExitEventArgs e)
{
    // 1. Stop API server (có thể mất vài giây)
    _apiHostService?.StopAsync().GetAwaiter().GetResult(); // sync wait
    _apiHostService?.Dispose();

    // 2. Disconnect và dispose tất cả PLC connections
    var plcManager = _serviceProvider?.GetService<IPlcManager>();
    plcManager?.Dispose();

    // 3. Flush log buffer ra file
    Log.CloseAndFlush();

    base.OnExit(e);
}
```

`.GetAwaiter().GetResult()` là cách call async method từ sync context. Tránh dùng `.Wait()` vì có thể deadlock trong một số context. `.GetAwaiter().GetResult()` cũng đúng semantics hơn: nếu task throw exception, nó re-throw synchronously thay vì wrap trong `AggregateException`.

`Log.CloseAndFlush()` phải là dòng cuối — đảm bảo không mất log nào. Nếu call trước, các logs từ các `Dispose()` calls sau sẽ bị mất.

10. LAYER 8 — VIEWS

Views/LoginWindow.xaml.cs

```
public LoginWindow(UserService userService)
{
    InitializeComponent();
    _userService = userService;

    LoadSavedCredentials();

    // Chromeless window – phải tự xử lý drag
    MouseLeftButtonDown += (_, e) =>
    {
        if (e.ButtonState == MouseButtonState.Pressed) DragMove();
    };

    Loaded += (_, _) =>
    {
        // Focus vào đúng field sau khi load
        if (string.IsNullOrEmpty(UserBox.Text)) UserBox.Focus();
        else PasswordBox.Focus();
    };
}
```

`Loaded` event thay vì `MouseLeftButtonDown` làm focus — `Loaded` fires sau khi controls đã render và có thể nhận focus. Nếu `Focus()` trong constructor, control chưa visible nên sẽ không có tác dụng.

Password show/hide pattern:

```
private void ShowPwdBtn_Changed(object sender, RoutedEventArgs e)
{
    bool show = ShowPwdBtn.IsChecked == true;
    if (show)
    {
        PasswordTextBox.Text = PasswordBox.Password; // copy sang TextBox
        PasswordBox.Visibility = Visibility.Collapsed;
        PasswordTextBox.Visibility = Visibility.Visible;
        PasswordTextBox.CaretIndex = PasswordTextBox.Text.Length; // cursor cuối
    }
    else
    {
        PasswordBox.Password = PasswordTextBox.Text; // copy sang PasswordBox
        PasswordTextBox.Visibility = Visibility.Collapsed;
        PasswordBox.Visibility = Visibility.Visible;
    }
}
```

Tại sao không chỉ dùng `PasswordBox.Password` binding? `PasswordBox.Password` không thể bind two-way qua XAML vì security — password không được lưu dưới dạng `string` (DependencyProperty) trong WPF để tránh bị inspect từ memory dump. Ta phải dùng code-behind.

SignIn_Click:

```
private void SignIn_Click(object sender, RoutedEventArgs e)
{
    HideError();
    var username = TextBox.Text?.Trim() ?? string.Empty;
    var password = GetPassword();

    if (string.IsNullOrEmpty(username)) { ShowError("Vui lòng nhập tên đăng nhập."); return; }
    if (string.IsNullOrEmpty(password)) { ShowError("Vui lòng nhập mật khẩu."); return; }

    LoginButton.IsEnabled = false; // prevent double-click

    try
    {
        LoggedInUser = _userService.ValidateCredentials(username, password);

        if (LoggedInUser != null)
        {
            if (RememberMeCheckBox.IsChecked == true) SaveCredentials(username);
            else ClearSavedCredentials();

            DialogResult = true; // signal thành công cho ShowDialog() caller
            Close();
        }
        else
        {
            ShowError("Tên đăng nhập hoặc mật khẩu không đúng.");
            ClearPassword();
            PasswordBox.Focus();
        }
    }
    catch (Exception ex)
    {
        Logger.Error(ex, "Login error");
        ShowError($"Đăng nhập thất bại: {ex.Message}");
    }
    finally
    {
        LoginButton.IsEnabled = true; // always re-enable
    }
}
```

`username.Trim()` — xóa whitespace đầu/cuối. User thường copy-paste username có trailing space. `ClearPassword()` sau login thất bại — không để password còn trong ô sau khi nhập sai.

SavedCredentials — không bao giờ lưu password:

```
public class SavedCredentials
{
    public string Username { get; set; } = string.Empty;
    // Không có Password field
}
```

Lưu username thôi — user chỉ cần nhập password mỗi lần (security tradeoff). Lưu password thậm chí encrypted vẫn là bad practice cho desktop app.

Views/MainWindow.xaml — Context Menu ngoài visual tree

Vấn đề: `ContextMenu` là `Popup` — nó không nằm trong visual tree của `ListBox`. Khi XAML binding `{Binding ConnectSelectedCommand}`, WPF traverse visual tree lên tìm `DataContext` nhưng dừng lại ở visual tree boundary (Popup wall).

Giải pháp: Tag + PlacementTarget:

```
<!-- Trong ListBox.ItemContainerStyle -->
<Setter Property="Tag"
        Value="{Binding DataContext, RelativeSource={RelativeSource AncestorType=ListBox}}"/>
<EventSetter Event="PreviewMouseRightButtonDown" Handler="PlcItem_RightClick"/>
<Setter Property="ContextMenu">
    <Setter.Value>
        <ContextMenu DataContext="{Binding PlacementTarget.Tag,
                                RelativeSource={RelativeSource Self}}">
            <MenuItem Header="  Kết nối"    Command="{Binding ConnectSelectedCommand}"/>
            <MenuItem Header="⌘ Ngắt kết nối" Command="{Binding DisconnectSelectedCommand}"/>
            <Separator/>
            <MenuItem Header="🌐 Browse Server" Command="{Binding BrowseServerCommand}"/>
            <Separator/>
            <MenuItem Header="✎ Sửa"    Command="{Binding EditPlcCommand}"/>
            <MenuItem Header="🗑 Xóa"    Command="{Binding DeletePlcCommand}"/>
        </ContextMenu>
    </Setter.Value>
</Setter>
```

1. `Tag = MainViewModel` — `ListBoxItem.Tag` lưu reference đến `MainViewModel` (lấy từ `ListBox.DataContext`)
2. `ContextMenu.PlacementTarget` = `ListBoxItem` mà menu được hiển thị từ đó
3. `PlacementTarget.Tag` = `MainViewModel` đã lưu ở bước 1
4. Commands trong MenuItem bind vào `MainViewModel`

PreviewMouseRightButtonDown để auto-select:

```
private void PlcItem_RightClick(object sender, MouseButtonEventArgs e)
{
    if (sender is ListBoxItem item)
        item.IsSelected = true;    // select item trước khi hiển thị ContextMenu
}
```

WPF không auto-select ListBoxItem khi right-click (chỉ left-click mới select). Nếu không có handler này, user right-click PLC2 nhưng PLC1 vẫn selected → ContextMenu commands tác động lên PLC1 sai.

`PreviewMouseRightButtonDown` thay vì `MouseRightButtonDown` — `Preview` events (tunnel từ root xuống) fire trước regular events (bubble từ element lên). Đảm bảo item được select trước khi ContextMenu nhận sự kiện.

Views/BrowseServerWindow.xaml.cs

```
public partial class BrowseServerWindow : Window
{
    private readonly BrowseServerViewModel _viewModel;

    public BrowseServerWindow(PlcConnection connection)
    {
        InitializeComponent();
        _viewModel = new BrowseServerViewModel(connection);
        DataContext = _viewModel;
    }

    // Expose tags được thêm cho caller (MainViewModel)
    public IReadOnlyList<TagItem> AddedTags => _viewModel.SelectedTags.ToList();

    private void TreeView_SelectedItemChanged(object sender, RoutedPropertyChangedEventArgs<object> e)
    {
        if (e.NewValue is BrowseNodeItem node)
            _viewModel.SelectedNode = node;
    }
}
```

`TreeView_SelectedItemChanged` — TreeView có riêng event `SelectedItemChanged` với `RoutedPropertyChangedEventArgs<object>`. `e.NewValue as BrowseNodeItem` safe-cast — null nếu không đúng type (placeholder items).

```

private void CloseButton_Click(object sender, RoutedEventArgs e)
{
    DialogResult = _viewModel.SelectedTags.Any(); // true nếu có tag nào được thêm
    Close();
}

protected override void OnMouseDoubleClick(MouseButtonEventArgs e)
{
    base.OnMouseDoubleClick(e);
    if (_viewModel.SelectedNode?.CanAddAsTag == true)
        _viewModel.AddSelectedTagCommand.Execute(null);
}
}

```

`OnMouseDoubleClick` override — double click trên TreeView node nào có `CanAddAsTag=true` thì add ngay, nhanh hơn phải click nút "Thêm tag". UX improvement.

11. CÁC BUG ĐÃ FIX

Bug 1: `NodeId.Parse` vs `new NodeId`

Vấn đề: `new NodeId("i=85")` tạo `NodeId` sai kiểu.

```

new NodeId("i=85") → Type=String, Identifier="i=85" ← SAI
NodeId.Parse("i=85") → Type=Numeric, Identifier=85 ← ĐÚNG

```

OPC UA server từ chối String `NodeId` cho nodes chuẩn (Objects, Types, Views folders).
 Kết quả: `BrowseAsync()` trả về empty list không có lỗi — khó debug.

Fix: Đổi `new NodeId(nodeId)` → `NodeId.Parse(nodeId)` trong `BrowseAsync()` và `GetNodeInfoAsync()`.

Bug 2: `BrowseServerViewModel` gọi trực tiếp OPC UA SDK

Vấn đề: `BrowseServerViewModel` trước đây inject `Session` trực tiếp và gọi `_session.Browse()`, `_session.ReadNode()`. Điều này: - Bypass service layer (`PlcConnection`)
 - Không có error handling, logging - `PlcConnection` có thể đang reconnect → session thay đổi mà `ViewModel` không biết

Fix: Đổi constructor nhận `PlcConnection` thay vì `Session`. Dùng `_connection.BrowseAsync()`, `_connection.GetNodeInfoAsync()`, `_connection.ReadTagAsync()`.

Bug 3: ContextMenu binding thất bại

Vấn đề: `ContextMenu` là Popup nằm ngoài visual tree. `RelativeSource AncestorType=Window` hoặc `AncestorType=ListBox` không traverse qua visual tree boundary → binding fail silently (command = null → exception hoặc không làm gì).

Fix: Tag pattern — lưu ViewModel vào `Tag` property của `ListBoxItem`, `ContextMenu` bind vào `PlacementTarget.Tag`.

12. PATTERNS VÀ QUYẾT ĐỊNH KIẾN TRÚC

Pattern 1: MVVM với WPF Binding

Vì sao dùng MVVM thay vì code-behind thuần?

Code-behind thuần:

Button.Click → fetch data → update TextBox.Text trực tiếp
→ logic trộn lẫn với UI, khó test, khó maintain

MVVM:

Button → Command → ViewModel method → update property → Binding → TextBox tự update
→ logic tách biệt, có thể unit test ViewModel mà không cần UI

Pattern 2: ObservableCollection cho Live Updates

```
public ObservableCollection<PlcDevice> PlcDevices { get; } = new();  
public ObservableCollection<LogEntry> LogEntries { get; } = new();
```

`ObservableCollection<T>` implements `INotifyCollectionChanged` — khi `Add()` hoặc `Remove()`, `ListBox/DataGrid` tự cập nhật mà không cần reassign `ItemsSource`. `List<T>` không có mechanism này.

Pattern 3: Fire-and-Forget với `_ =`

```
_ = Task.Run(async () => { ... }); // background work  
_ = InitializeViewModelAsync(_mainViewModel); // startup task  
_ = LoadRootNodesAsync(); // ViewModel init
```

Dấu `_` (discard) tắt warning "CS4014: Because this call is not awaited...". Không phải ignore exception — vẫn cần handle trong task body. Dùng khi: tác vụ phụ không ảnh hưởng luồng chính, hoặc không cần chờ kết quả.

Pattern 4: `private static ILogger Logger => Log.Logger`

Dùng cho các class được tạo trước khi UiSink được add (PlcConnection, PlcManager). Static property thay vì field — mỗi lần gọi lấy logger hiện tại, không phải instance cũ đã captured.

Pattern 5: Atomic file write (temp + rename)

```
File.WriteAllText(tempPath, json); // write vào .tmp
File.Delete(targetPath);
File.Move(tempPath, targetPath); // rename – atomic trên OS level
```

Tránh corrupt file khi app crash giữa write. Dùng cho lock_state.json và có thể áp dụng cho config file.

Pattern 6: Double-check locking

```
if (_reconnectHandler == null) // check ngoài lock (fast path)
{
    lock (_lock)
    {
        if (_reconnectHandler == null) // check lại trong lock (safe)
        {
            _reconnectHandler = new SessionReconnectHandler(...);
        }
    }
}
```

Tối ưu performance: kiểm tra không lock trước (99% cases). Chỉ lock và kiểm tra lại khi `null`. Tránh overhead của lock khi không cần.

Pattern 7: CancellationToken trên async methods

```
public async Task<bool> ConnectAsync(CancellationToken cancellationToken = default)
```

`default` = `CancellationToken.None` — không cancellable nếu caller không truyền token.

Dùng khi: user click Cancel trong lúc đang connect → cancel operation. Propagate token xuống `Session.Create()`, `Task.Delay()`, v.v.

Pattern 8: IReadOnlyList return type

```
public async Task<IReadOnlyList<BrowseNode>> BrowseAsync(...) { ... }
```

Trả về `IReadOnlyList<T>` thay vì `List<T>` — caller không thể `.Add()` hay `.Remove()` vào list trả về. Tránh side effects. Caller phải tạo copy riêng nếu muốn modify.

TỔNG KẾT

Dự án OPC UA Communication Engine được xây dựng theo kiến trúc phân lớp rõ ràng:

Lớp	Trách nhiệm	Files chính
Enums	Kiểu dữ liệu liệt kê	Enums/*.cs
Models	Data transfer + binding	Models/*.cs
Interfaces	Contracts giữa layers	Interfaces/*.cs
Helpers	Utilities tái sử dụng	Helpers/*.cs
Services	Business logic + I/O	Services/*.cs
ViewModels	UI state + commands	ViewModels/*.cs
Views	XAML + event handlers	Views/*.xaml.cs
App	DI wiring + startup	App.xaml.cs

Nguyên tắc cốt lõi: 1. **Dependency flows one way** — View → ViewModel → Service → Model. Không bao giờ ngược lại. 2. **UI thread safety** — Mọi thay đổi UI phải qua `Dispatcher`. `ObservableObject` tự handle. 3. **Fail gracefully** — Self-healing configs, BCrypt try/catch, connection retry với exponential backoff. 4. **Separation of concerns** — PlcConnection chỉ biết OPC UA. PlcManager chỉ biết quản lý connections. ViewModel chỉ biết UI state. 5. **Logs everywhere** — Serilog với structured logging ở mọi critical path, UI sink cho operator.

Tài liệu này được tạo tự động từ source code của dự án OPCUACommDriver.
Ngày: 2026-06-19

Chương 5: Helpers - RelayCommand, AsyncRelayCommand, RelayCommand<T>

5.1 Tổng quan về ICommand Interface

Trong WPF (Windows Presentation Foundation), giao tiếp giữa UI và business logic được thực hiện thông qua pattern **MVVM (Model-View-ViewModel)**. Một trong những thành phần cốt lõi của MVVM chính là **ICommand interface**.

`ICommand` được định nghĩa trong namespace `System.Windows.Input` và chứa ba thành viên bắt buộc:

```
public interface ICommand
{
    event EventHandler? CanExecuteChanged;
    bool CanExecute(object? parameter);
    void Execute(object? parameter);
}
```

Tại sao WPF cần ICommand?

Trong WPF, các Button, MenuItem, và nhiều control khác có một property tên là `Command`. Khi user click button, WPF không gọi event handler trực tiếp - thay vào đó nó gọi `command.Execute()`. Điều này có những ưu điểm:

1. **Separation of Concerns:** Logic nằm trong ViewModel, không phải code-behind của View
2. **Enable/Disable tự động:** WPF gọi `CanExecute()` định kỳ để tự động enable/disable button
3. **Testability:** ViewModel (và command) có thể được unit test mà không cần UI

Thay vì implement ICommand cho từng action, `RelayCommand` đóng vai trò là một implementation tái sử dụng, nhận `Action` và `Func<bool>` qua constructor.

5.2 RelayCommand - Implementation chi tiết

```
using System.Windows.Input;

namespace OpcUaCommunicationEngine.Helpers;

public class RelayCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Func<object?, bool>? _canExecute;
```

Dòng 1: `using System.Windows.Input;` - import namespace chứa `ICommand` và `CommandManager`.

Dòng 5-6: Hai private readonly field: - `_execute`: Delegate kiểu `Action<object?>` - đây là logic sẽ chạy khi command được thực thi. `object?` vì `ICommand.Execute` nhận `object?`. - `_canExecute`: Delegate kiểu `Func<object?, bool>?` - nullable, nếu null thì command luôn enabled. Nhận parameter và trả về bool.

5.2.1 CanExecuteChanged và CommandManager.RequerySuggested

```
public event EventHandler? CanExecuteChanged
{
    add => CommandManager.RequerySuggested += value;
    remove => CommandManager.RequerySuggested -= value;
}
```

Đây là custom event accessor - thay vì tạo event riêng, chúng ta **delegate** (ủy thác) lên `CommandManager.RequerySuggested`.

CommandManager.RequerySuggested là gì?

`CommandManager` là static class của WPF, theo dõi trạng thái của UI. `RequerySuggested` là event mà WPF bắn ra khi nó "nghĩ ngờ" rằng `CanExecute` của các command có thể đã thay đổi - ví dụ khi: - User thay đổi focus - User nhập text - Một UI element thay đổi trạng thái

Khi event này bắn, WPF sẽ gọi lại `CanExecute()` trên tất cả các command đang được binding, và tự động enable/disable các control tương ứng.

Tại sao dùng add/remove accessor thay vì event thường?

Nếu chúng ta viết:

```
public event EventHandler? CanExecuteChanged;
```

thì WPF sẽ không biết khi nào cần refresh - chúng ta phải tự raise event này.

Bằng cách delegate lên `CommandManager.RequerySuggested`, chúng ta để WPF tự quản lý việc này một cách thông minh. Mỗi khi WPF gửi `RequerySuggested`, tất cả các button/control đang bind command này sẽ tự động cập nhật trạng thái.

5.2.2 Constructor và Null Guard

```
public RelayCommand(Action<object?> execute, Func<object?, bool>? canExecute = null)
{
    _execute = execute ?? throw new ArgumentNullException(nameof(execute));
    _canExecute = canExecute;
}
```

Dòng `_execute = execute ?? throw new ArgumentNullException(nameof(execute));`

Toán tử `??` (null-coalescing) ở đây được dùng như một **null guard**: - Nếu `execute` không null: gán vào `_execute` - Nếu `execute` null: throw `ArgumentNullException` với tên parameter

Đây là pattern bảo vệ an toàn - `RelayCommand` mà không có `execute` delegate thì hoàn toàn vô nghĩa. Fail nhanh tại constructor còn hơn là crash bí ẩn sau này khi `Execute()` được gọi với null delegate.

`nameof(execute)` trả về string `"execute"` - dùng cách này thay vì hard-code string để nếu rename parameter, compiler sẽ báo lỗi thay vì silently break.

```
public RelayCommand(Action execute, Func<bool>? canExecute = null)
    : this(_ => execute(), canExecute != null ? _ => canExecute() : null)
{
}
```

Constructor overload thứ hai - convenience constructor cho trường hợp command không cần parameter: - `_ => execute()` : lambda nhận parameter nhưng bỏ qua (dấu `_` là convention cho "discard") - `canExecute != null ? _ => canExecute() : null` : nếu có `canExecute`, wrap nó thành `Func<object?, bool>` ; nếu không thì null

`:this(...)` gọi constructor chính - tránh duplicate code.

5.2.3 CanExecute và Execute

```
public bool CanExecute(object? parameter) => _canExecute == null || _canExecute(parameter);
public void Execute(object? parameter) => _execute(parameter);
public void RaiseCanExecuteChanged() => CommandManager.InvalidateRequerySuggested();
```

CanExecute: Short-circuit evaluation: - Nếu `_canExecute == null` → true (không có điều kiện = luôn enabled) - Nếu có `_canExecute` → gọi delegate với parameter

Execute: Đơn giản gọi `_execute`. Không có null check vì đã guard trong constructor.

RaiseCanExecuteChanged: Gọi `CommandManager.InvalidateRequerySuggested()` - báo cho WPF biết cần re-evaluate `CanExecute` của tất cả commands. ViewModel gọi method này khi state thay đổi mà WPF không tự phát hiện được.

5.3 AsyncRelayCommand - Xử lý bất đồng bộ

```
public class AsyncRelayCommand : ICommand
{
    private readonly Func<Task> _execute;
    private readonly Func<bool>? _canExecute;
    private bool _isExecuting;
```

Sự khác biệt then chốt: `_execute` là `Func<Task>` thay vì `Action<object?>`. Command này dành cho các operation async như kết nối PLC, load dữ liệu từ file, v.v.

5.3.1 IsExecuting - Ngăn Re-entry

```
public bool IsExecuting
{
    get => _isExecuting;
    private set { _isExecuting = value; RaiseCanExecuteChanged(); }
}
```

Property `IsExecuting` có setter private - chỉ code trong class mới set được.

Tại sao cần IsExecuting?

Khi user click button kích hoạt một async operation (ví dụ kết nối OPC UA mất 3-5 giây), nếu không có guard, user có thể click liên tục và tạo ra nhiều connection attempts song song - gây race condition, resource leak, hoặc crash.

Cơ chế: 1. `IsExecuting = true` → setter gọi `RaiseCanExecuteChanged()` 2. WPF re-evaluate `CanExecute()` → trả về false vì `_isExecuting = true` 3. Button bị disable → user không click được nữa 4. Operation hoàn thành → `IsExecuting = false` → button enable lại

5.3.2 Tại sao Execute là `async void` chứ không phải `async Task` ?

```
public async void Execute(object? parameter)
{
    if (!CanExecute(parameter)) return;
    try { IsExecuting = true; await _execute(); }
    finally { IsExecuting = false; }
}
```

Đây là câu hỏi quan trọng về C# và WPF.

`ICommand.Execute` được định nghĩa trả về `void` :

```
void Execute(object? parameter);
```

Vì interface signature là `void` , chúng ta **buộc phải** implement với `void` . Không thể đổi thành `Task` vì vi phạm interface contract.

`async void` trong C# có những đặc điểm: - Exception không được propagate ra caller (caller không thể await) - Nếu có exception, nó sẽ bị throw vào SynchronizationContext (UI thread) - Thường được coi là anti-pattern, **ngoại trừ** event handlers và `ICommand.Execute`

Để bù đắp, `_execute` là `Func<Task>` - exception từ Task này sẽ propagate vào `async void Execute` , và nếu không được catch, sẽ crash ứng dụng (unhandled exception).

Khối `try/finally` đảm bảo `IsExecuting = false` **luôn luôn** được gọi dù có exception hay không.

5.4 RelayCommand<T> - Generic Version

```
public class RelayCommand<T> : ICommand
{
    private readonly Action<T?> _execute;
    private readonly Func<T?, bool>? _canExecute;

    ...

    public bool CanExecute(object? parameter) => _canExecute == null || _canExecute((T?)parameter);
    public void Execute(object? parameter) => _execute((T?)parameter);
}
```

`RelayCommand<T>` cho phép type-safe commands. Thay vì nhận `object?` và phải cast trong body của action, casting được thực hiện tại boundary của command.

Casting `(T?)parameter` :

`ICommand.Execute` và `ICommand.CanExecute` nhận `object?` - đây là constraint từ interface. Khi WPF pass `CommandParameter` vào, nó là `object?`.

`(T?)parameter` là **unboxing/casting** từ `object?` sang `T?`. Nếu `T` là reference type thì là downcast. Nếu `T` là value type thì là unboxing. Nếu type không khớp, sẽ throw `InvalidCastException`.

Ví dụ sử dụng:

```
// ViewModel
ConnectCommand = new RelayCommand<PlcDevice>(device => ConnectToPlc(device));

// XAML
<Button Command="{Binding ConnectCommand}" CommandParameter="{Binding SelectedDevice}"/>
```

WPF sẽ pass `SelectedDevice` (kiểu `PlcDevice`) vào `Execute`, command cast về `PlcDevice?`, action nhận `PlcDevice?` typed - không cần manual cast trong body.

5.5 Tổng kết Helpers Pattern

Class	Execute Type	Use Case
<code>RelayCommand</code>	<code>Action<object?></code>	Sync operations, no parameter type
<code>RelayCommand<T></code>	<code>Action<T?></code>	Sync operations, typed parameter
<code>AsyncRelayCommand</code>	<code>Func<Task></code>	Async operations, prevents re-entry

Ba class này là backbone của toàn bộ command binding trong ứng dụng OPC UA, cho phép ViewModel expose các operation ra cho View mà không cần code-behind.

Chương 6: Services/Auth - UserService và OperatorLockService

6.1 UserService - Quản lý người dùng

6.1.1 Khởi tạo và Default Admin

Constructor của `UserService` thực hiện hai việc quan trọng:

- Load danh sách users từ JSON file:** Đọc file `users.json` và deserialize thành `List<UserAccount>`
- Tạo default admin nếu chưa có:** Nếu file không tồn tại hoặc chưa có user nào có role Admin, tạo user `admin/admin123`

```
// Pseudo-code của constructor
public UserService(string dataFilePath)
{
    _dataFilePath = dataFilePath;
    LoadUsersFromFile();

    if (!_users.Any(u => u.Role == UserRole.Admin))
    {
        CreateDefaultAdmin();
    }
}
```

Tại sao tạo default admin?

Đây là pattern phổ biến trong ứng dụng enterprise: lần đầu chạy, hệ thống cần ít nhất một account để vào cấu hình. Default `admin/admin123` là well-known credentials mà admin thực tế phải đổi ngay.

6.1.2 Thread Safety với lock(_lock)

```
private readonly object _lock = new();

public IEnumerable<UserAccount> GetAllUsers()
{
    lock (_lock)
    {
        return _users.ToList(); // ToList() để trả về copy, không expose internal list
    }
}
```

Tại sao cần lock trên mọi read operation?

Ứng dụng này có nhiều thread đồng thời: - **UI thread**: ViewModel gọi UserService để hiển thị danh sách users - **API thread (ASP.NET)**: HTTP request validate credentials - **Timer thread**: Auto-logout, session cleanup

Nếu không có lock, có thể xảy ra **race condition**: - Thread A đang đọc `_users` list - Thread B đang modify list (add/remove user) - Thread A đọc được list ở trạng thái corrupt

`lock(_lock)` đảm bảo chỉ một thread vào critical section tại một thời điểm. `_lock` là plain `object` - đây là pattern chuẩn cho locking.

`ToList()` tạo copy của internal list trước khi trả về - tránh caller giữ reference đến mutable internal collection.

6.1.3 BCrypt và Security Decisions

```
private string HashPassword(string password)
    => BCrypt.HashPassword(password, workFactor: 11);

private bool VerifyPassword(string password, string hash)
{
    try { return BCrypt.Verify(password, hash); }
    catch { return false; }
}
```

Tại sao BCrypt với workFactor=11?

BCrypt là password hashing algorithm được thiết kế đặc biệt để **chậm**. `workFactor=11` có nghĩa là $2^{11} = 2048$ iterations, mất khoảng 100-200ms trên CPU hiện đại.

So sánh với MD5/SHA256: - SHA256: < 1 microsecond per hash → attacker có thể thử hàng tỷ passwords/giây - BCrypt(11): ~150ms per hash → attacker chỉ thử được ~6 passwords/giây

Với database 1000 users bị leak, attacker cần thời gian **vô cùng lớn** để brute force.

Tại sao wrap trong try/catch?

`BCrypt.Verify` có thể throw exception nếu hash string bị corrupt (ví dụ truncated trong database). Thay vì để exception propagate lên, chúng ta return `false` - "password không hợp lệ". Đây là defensive programming: invalid hash = failed verification.

6.1.4 Username Normalization

```
public async Task<UserAccount> CreateUser(string username, ...)
{
    // Kiểm tra unique, case-insensitive
    if (_users.Any(u => string.Equals(u.Username, username, StringComparison.OrdinalIgnoreCase)))
        throw new InvalidOperationException("Username already exists");

    // Lưu lowercase
    var newUser = new UserAccount
    {
        Username = username.ToLowerInvariant(),
        ...
    };
}
```

OrdinalIgnoreCase: So sánh string theo byte value, không phụ thuộc culture, case-insensitive. Dùng cho username vì `"Admin"`, `"ADMIN"`, `"admin"` đều phải là cùng một user.

ToLowerInvariant(): Normalize về lowercase, dùng invariant culture (không phụ thuộc locale). Ví dụ: tiếng Thổ Nhĩ Kỳ có 'İ' uppercase → 'i' lowercase (khác ASCII 'i').

`ToLowerInvariant()` luôn dùng ASCII rules.

Lưu lowercase nhưng compare case-insensitive → đảm bảo consistency: dù user nhập `"Admin"` hay `"ADMIN"`, system luôn tìm thấy `"admin"` trong database.

6.1.5 UpdateUser - Partial Update Pattern

```
public UserAccount UpdateUser(string userId, string? newDisplayName, string? newEmail, UserRole? newRole)
{
    lock (_lock)
    {
        var user = GetUserById(userId) ?? throw new KeyNotFoundException();

        if (newDisplayName != null) user.DisplayName = newDisplayName;
        if (newEmail != null) user.Email = newEmail;
        if (newRole != null) user.Role = newRole.Value;

        SaveUsersToFile();
        return user;
    }
}
```

Partial update - chỉ cập nhật field nào được truyền vào (khác null). Điều này cho phép:

- Client chỉ gửi fields cần thay đổi - Fields không gửi giữ nguyên giá trị cũ

Nếu dùng full replace (ghi đè toàn bộ object), client phải gửi đầy đủ mọi field, để vô tình xóa dữ liệu.

6.1.6 DeleteUser - Ngăn Xóa Admin Cuối

```
public void DeleteUser(string userId)
{
    lock (_lock)
    {
        var user = GetUserById(userId) ?? throw new KeyNotFoundException();

        if (user.Role == UserRole.Admin)
        {
            var adminCount = _users.Count(u => u.Role == UserRole.Admin && u.IsActive);
            if (adminCount <= 1)
                throw new InvalidOperationException("Cannot delete the last active admin");
        }

        _users.Remove(user);
        SaveUsersToFile();
    }
}
```

Tại sao check last admin?

Nếu xóa admin cuối cùng, hệ thống sẽ không có cách nào để vào admin panel. Đây là **safety guard** - ngăn admin tự "lock mình ra ngoài".

Check `IsActive` vì admin bị disable cũng không login được - cần tính cả active admins.

6.2 OperatorLockService - Kiểm soát quyền Write

6.2.1 Mục đích và Design

Trong môi trường công nghiệp, nhiều operator có thể cùng monitor PLC nhưng **chỉ một người được phép write** tại một thời điểm. Viết đồng thời từ nhiều operator có thể gây:

- Race condition trên thiết bị
- Conflicting commands
- Safety hazards

`OperatorLockService` implement **pessimistic locking**: operator phải "claim" lock trước khi write. Lock có timeout để tự động expire nếu operator quên release.

State được persist vào `lock_state.json` để survive application restart.

6.2.2 TryAcquireLock - Logic chi tiết

```
public LockAcquireResult TryAcquireLock(
    string userId, string username, string displayName,
    UserRole role, int durationMinutes)
{
    lock (_lock)
    {
        // 1. Role check
        if (role < UserRole.Operator)
            return LockAcquireResult.Denied("Insufficient role");

        // 2. Same user có lock chưa expired → extend
        if (_currentLock?.UserId == userId && !IsLockExpired(_currentLock))
            return ExtendLock(durationMinutes);

        // 3. Khác user đang giữ lock → deny với holder info
        if (_currentLock != null && !IsLockExpired(_currentLock))
            return LockAcquireResult.Denied($"Held by {_currentLock.Username}");

        // 4. Tính expiry
        DateTime expiresAt;
        if (role >= UserRole.Admin || durationMinutes == 0)
            expiresAt = DateTime.MaxValue; // Unlimited
        else
            expiresAt = DateTime.UtcNow.AddMinutes(
                Math.Min(durationMinutes, MaxLockDurationMinutes));

        // 5. Tạo lock mới và atomic write
        _currentLock = new LockState { UserId = userId, ... ExpiresAt = expiresAt };
        AtomicWriteLockState();
        return LockAcquireResult.Success(_currentLock);
    }
}
```

Role check: `role < UserRole.Operator` - enum comparison. Viewer và Guest không được acquire lock.

Same user extend: Nếu operator đang giữ lock và request lại, thay vì deny, chúng ta extend thời gian. Điều này tránh operator bị interrupt giữa chừng vì lock expire.

DateTime.MaxValue cho unlimited: Admin không bị limit thời gian. `durationMinutes=0` cũng là tín hiệu "unlimited". `DateTime.MaxValue` là `9999-12-31` - thực tế là vĩnh viễn.

MaxLockDurationMinutes: Cap duration để ngăn operator giữ lock quá lâu. Ví dụ max 8 giờ - nếu operator quên release, system tự release sau 8 giờ.

6.2.3 Atomic File Write

```
private void AtomicWriteLockState()
{
    var tmpPath = _lockFilePath + ".tmp";
    var json = JsonConvert.SerializeObject(_currentLock, Formatting.Indented);

    File.WriteAllText(tmpPath, json);          // Ghi vào .tmp
    File.Move(tmpPath, _lockFilePath, true); // Rename atomic
}
```

Tại sao cần atomic write?

Nếu application crash giữa chừng khi đang ghi file: - File ghi một nửa → JSON corrupt -
Lần sau load → parse error → lock state mất

Giải pháp: Write to temp file (`lock_state.json.tmp`), sau đó rename. Rename là **atomic operation** trên hầu hết filesystems - file hoặc là file cũ hoặc là file mới, không có trạng thái trung gian.

`File.Move(tmpPath, _lockFilePath, true)` - parameter `true` là overwrite nếu destination đã tồn tại (C# 8.0+).

6.2.4 FileSystemWatcher - Multi-Instance Support

```
private void StartFileWatcher()
{
    _watcher = new FileSystemWatcher(Path.GetDirectoryName(_lockFilePath)!)
    {
        Filter = Path.GetFileName(_lockFilePath),
        NotifyFilter = NotifyFilters.LastWrite | NotifyFilters.FileName
    };

    _watcher.Changed += OnLockFileChanged;
    _watcher.EnableRaisingEvents = true;
}

private void OnLockFileChanged(object sender, FileSystemEventArgs e)
{
    if (_isInternalUpdate) return; // Bỏ qua self-triggered events

    Task.Delay(100).ContinueWith(_ => ReloadLockStateFromFile());
}
```

Tại sao cần FileSystemWatcher?

Trong môi trường production, có thể chạy nhiều instances của ứng dụng trên cùng máy (ví dụ: primary và backup). Nếu instance A acquire lock và ghi file, instance B cần biết về thay đổi này.

FileSystemWatcher monitor file system events và notify khi `lock_state.json` thay đổi từ bên ngoài.

`_isInternalUpdate` flag: Khi chính ứng dụng ghi file (atomic write), FileSystemWatcher cũng sẽ raise event `Changed`. Nếu không có guard, chúng ta sẽ reload file mà chính mình vừa ghi → vô hại nhưng lãng phí. `_isInternalUpdate = true` trước khi ghi, `= false` sau khi ghi xong.

100ms delay: Delay nhỏ trước khi reload - đảm bảo file đã được ghi xong hoàn toàn trước khi read. FileSystemWatcher event có thể fire ngay khi file bắt đầu được ghi, chưa có đủ dữ liệu.

6.2.5 StartTimeoutChecker - Background Timer

```
private void StartTimeoutChecker()
{
    _timeoutTimer = new System.Threading.Timer(
        callback: _ => CheckAndExpireLock(),
        state: null,
        dueTime: TimeSpan.FromSeconds(30),
        period: TimeSpan.FromSeconds(30));
}

private void CheckAndExpireLock()
{
    lock (_lock)
    {
        if (_currentLock != null && IsLockExpired(_currentLock))
        {
            var expiredLock = _currentLock;
            _currentLock = null;
            AtomicWriteLockState();
            LockExpired?.Invoke(this, expiredLock);
        }
    }
}
```

Timer chạy mỗi 30 giây, check nếu lock đã expired. Nếu expired: 1. Xóa lock khỏi memory 2. Ghi file (null lock state) 3. Raise event `LockExpired` để notify UI

30 giây là balance giữa responsiveness (lock expire nhanh) và performance (không check quá thường xuyên).

Chương 7: Services - ConfigurationService, DataCacheService, UiLogSink

7.1 UiLogSink - Đưa Log vào UI

7.1.1 Kiến trúc Logging

Ứng dụng dùng **Serilog** làm logging framework. Serilog có concept "sinks" - đầu ra của log. Mặc định có FileSink, ConsoleSink. `UiLogSink` là custom sink để hiển thị log trực tiếp trong UI.


```
public class UiLogSink : ILogEventSink
{
    private readonly ObservableCollection<LogEntry> _logEntries;
    private readonly System.Windows.Threading.Dispatcher _dispatcher;
    private readonly int _maxEntries;
```

ILogEventSink: Interface của Serilog, yêu cầu implement method `Emit(LogEvent)` .

ObservableCollection<LogEntry>: Collection đặc biệt trong WPF - tự động notify UI khi có item được add/remove. ListView/DataGrid binding lên collection này sẽ tự cập nhật.

Dispatcher: Object đại diện cho UI thread của WPF. Mọi thay đổi UI phải chạy trên UI thread.

7.1.2 Constructor - Capture Dispatcher

```
public UiLogSink(ObservableCollection<LogEntry> logEntries, int maxEntries = 1000)
{
    _logEntries = logEntries;
    _dispatcher = System.Windows.Application.Current.Dispatcher;
    _maxEntries = maxEntries;
}
```

`Application.Current.Dispatcher` - lấy Dispatcher của UI thread. **Phải** gọi trong constructor khi còn đang trên UI thread (vì `UiLogSink` được tạo trong App.xaml.cs trên UI thread).

Nếu không capture lúc này, sau này `Emit()` có thể được gọi từ background thread và không có cách nào lấy UI thread dispatcher.

`_maxEntries = 1000` (default) - giới hạn số log entries để tránh memory leak. Log cứ accumulate mãi sẽ làm UI chậm và dùng nhiều RAM.

7.1.3 Emit - Thread-Safe UI Update

```
public void Emit(LogEvent logEvent)
{
    var entry = new LogEntry
    {
        Timestamp = logEvent.Timestamp.LocalDateTime,
        Level = GetLevelShortName(logEvent.Level),
        Message = logEvent.RenderMessage(),
        Exception = logEvent.Exception?.ToString()
    };

    _dispatcher.InvokeAsync(() =>
    {
        _logEntries.Add(entry);
        while (_logEntries.Count > _maxEntries)
            _logEntries.RemoveAt(0);
    });
}
```

Serilog gọi Emit() từ thread nào?

Emit() có thể được gọi từ bất kỳ thread nào đã log - background service, timer thread, API thread. Nhưng modify `_logEntries` (ObservableCollection) từ non-UI thread sẽ throw `InvalidOperationException`.

`_dispatcher.InvokeAsync()`: Marshal việc modify collection sang UI thread.

`InvokeAsync` (khác với `Invoke`) là fire-and-forget - Emit() return ngay mà không đợi UI thread process. Điều này tránh deadlock và blocking logger.

Trimming strategy:

```
while (_logEntries.Count > _maxEntries)
    _logEntries.RemoveAt(0);
```

`while` thay vì `if` - để handle trường hợp nhiều log entries được batch add. RemoveAt(0) xóa entry cũ nhất (FIFO - oldest out).

7.1.4 GetLevelShortName - Serilog Level Abbreviation

```
private static string GetLevelShortName(LogEventLevel level) => level switch
{
    LogEventLevel.Verbose => "VRB",
    LogEventLevel.Debug => "DBG",
    LogEventLevel.Information => "INF",
    LogEventLevel.Warning => "WRN",
    LogEventLevel.Error => "ERR",
    LogEventLevel.Fatal => "FTL",
    _ => "???"
};
```

Switch expression (C# 8.0) - concise thay vì if/else chain. Trả về 3-character abbreviation phù hợp với Serilog convention (`{Level:u3}`).

7.1.5 UiLogSinkExtensions - Fluent API

```
public static class UiLogSinkExtensions
{
    public static LoggerConfiguration UiSink(
        this LoggerSinkConfiguration sinkConfig,
        ObservableCollection<LogEntry> logEntries,
        int maxEntries = 1000)
        => sinkConfig.Sink(new UiLogSink(logEntries, maxEntries));
}
```

Extension method cho `LoggerSinkConfiguration` - cho phép dùng fluent syntax khi configure Serilog:

```
Log.Logger = new LoggerConfiguration()
    .WriteTo.File("app.log")
    .WriteTo.UiSink(LogEntries) // Extension method của chúng ta
    .CreateLogger();
```

`this LoggerSinkConfiguration sinkConfig` - đây là extension method, sinkConfig là object được extend.

7.2 ConfigurationService - Quản lý cấu hình ứng dụng

7.2.1 LoadConfigurationAsync

```
public async Task<AppConfiguration> LoadConfigurationAsync()
{
    if (!File.Exists(_configFilePath))
    {
        var defaultConfig = CreateDefaultConfiguration();
        await SaveConfigurationAsync(defaultConfig);
        return defaultConfig;
    }

    var json = await File.ReadAllTextAsync(_configFilePath);
    var config = JsonConvert.DeserializeObject<AppConfiguration>(json, GetJsonSettings());

    return config ?? CreateDefaultConfiguration();
}
```

Flow: 1. File không tồn tại → tạo default config, lưu lại, return 2. File tồn tại → đọc JSON → deserialize → return 3. JSON null (file rỗng hoặc "null") → fallback về default

`await File.ReadAllTextAsync` - async I/O, không block UI thread trong khi đọc file.

7.2.2 SaveConfigurationAsync và JSON Settings

```
private static JsonSerializerSettings GetJsonSettings() => new JsonSerializerSettings
{
    Formatting = Formatting.Indented,
    NullValueHandling = NullValueHandling.Ignore,
    DateFormatString = "yyyy-MM-dd HH:mm:ss"
};
```

Formatting.Indented: JSON output có indent (pretty print), dễ đọc khi mở file bằng text editor.

NullValueHandling.Ignore: Không serialize properties có giá trị null. Giúp JSON gọn hơn và tránh confusion khi null nghĩa là "not set" vs "set to null".

DateFormatString: Override default ISO 8601 format của Newtonsoft. `"yyyy-MM-dd HH:mm:ss"` là format thân thiện hơn cho non-technical users đọc config file. Tuy nhiên mất timezone info - cần cẩn thận nếu config được share cross-timezone.

7.2.3 ValidateConfiguration

```
public List<string> ValidateConfiguration(AppConfiguration config)
{
    var errors = new List<string>();

    // Check empty names
    if (config.PlcDevices.Any(d => string.IsNullOrEmpty(d.Name)))
        errors.Add("PLC device has empty name");

    // Check invalid URLs
    foreach (var device in config.PlcDevices)
    {
        if (!Uri.TryCreate(device.EndpointUrl, UriKind.Absolute, out _))
            errors.Add($"Invalid URL for device '{device.Name}': {device.EndpointUrl}");
    }

    // Check duplicate names
    var duplicates = config.PlcDevices
        .GroupBy(d => d.Name, StringComparer.OrdinalIgnoreCase)
        .Where(g => g.Count() > 1)
        .Select(g => g.Key);

    foreach (var dup in duplicates)
        errors.Add($"Duplicate device name: '{dup}'");

    return errors;
}
```

Validation return list of errors thay vì throw exception - cho phép collect **tất cả** errors và hiển thị một lần, thay vì fail on first error và user phải fix từng cái.

7.3 DataCacheService - In-Memory Cache

```
public class DataCacheService
{
    private readonly ConcurrentDictionary<string, TagValue> _cache = new();

    public void UpdateTagValue(string tagId, TagValue value)
        => _cache[tagId] = value;

    public TagValue? GetTagValue(string tagId)
        => _cache.TryGetValue(tagId, out var value) ? value : null;

    public IReadOnlyDictionary<string, TagValue> GetAllValues()
        => _cache;
}
```

ConcurrentDictionary: Thread-safe dictionary từ `System.Collections.Concurrent`. Cho phép multiple threads read/write đồng thời mà không cần manual lock.

Vai trò trong kiến trúc: - OPC UA Subscriptions update cache liên tục khi tag values thay đổi (ví dụ 100ms interval) - REST API endpoint serve latest values từ cache - UI binding cũng đọc từ cache

Nếu không có cache, mỗi API request phải query PLC trực tiếp → latency cao, tải nặng lên PLC network. Cache cho phép API respond trong microseconds.

Eventual consistency: Cache có thể lag 1 interval behind PLC real value. Với subscription interval 100ms, lag tối đa 100ms - chấp nhận được cho hầu hết use cases.

Chương 8: Services/OpcUa - PlcConnection và PlcManager

8.1 PlcConnection - Kết nối OPC UA

8.1.1 Key Fields và Ý Nghĩa

```
private readonly PlcDevice _device;
private readonly object _lock = new();
private Session? _session;
private ApplicationConfiguration? _appConfig;
private PlcConnectionState _connectionState = PlcConnectionState.Disconnected;
private CancellationTokenSource? _reconnectCts;
private bool _disposed;
private bool _isDisconnecting;
private bool _isReconnecting;
private SessionReconnectHandler? _reconnectHandler;
private DateTime _lastReconnectCompleteTime = DateTime.MinValue;
private const int KeepAliveSettlingPeriodMs = 2000;
private readonly ConcurrentDictionary<string, Subscription> _subscriptions = new();
private readonly ConcurrentDictionary<uint, (string TagId, string NodeId)> _monitoredItemMapping = new();
private static ILogger Logger => Log.Logger;
```

_device : Configuration của PLC này - endpoint URL, credentials, danh sách subscriptions.

_lock : Object dùng cho locking critical sections trong connect/disconnect flow.

_session : OPC UA Session object từ OPC Foundation SDK. Nullable vì chưa có session khi Disconnected.

_connectionState : Enum state machine - Disconnected, Connecting, Connected, Reconnecting, Error.

`_reconnectCts` : CancellationTokenSource để cancel reconnect loop khi disconnect được request.

`_disposed` : Flag cho IDisposable pattern.

`_isDisconnecting` : Guard để phân biệt intentional disconnect với connection loss.

`_isReconnecting` : Tránh start multiple reconnect attempts đồng thời.

`_reconnectHandler` : OPC UA SDK built-in handler cho Secure Channel reconnect.

`_lastReconnectCompleteTime` và `KeepAliveSettlingPeriodMs` : Sau khi reconnect xong, có thể có stale KeepAlive failure events trong queue. Settling period 2000ms ignore những events này.

`_subscriptions` : Dictionary mapping subscription group name → Subscription object.

`_monitoredItemMapping` : Dictionary mapping MonitoredItem handle (uint) → TagId và NodeId. Dùng để identify tag khi receive notification.

8.1.2 Logger là Property, không phải Field

```
// Property - đọc Log.Logger tại runtime
private static ILogger Logger => Log.Logger;

// vs Field - capture Log.Logger tại startup (WRONG pattern)
private static readonly ILogger _logger = Log.Logger;
```

Đây là một subtlety quan trọng của Serilog.

Serilog có static `Log.Logger` property. Khi app khởi động, `Log.Logger` ban đầu là `NullLogger` (không làm gì). Sau đó trong `App.xaml.cs`, chúng ta configure lại:

```
// App.xaml.cs OnStartup()
Log.Logger = new LoggerConfiguration()
    .WriteTo.File(...)
    .WriteTo.UiSink(LogEntries) // UI sink được add sau
    .CreateLogger();
```

Nếu `PlcConnection` capture `Log.Logger` vào field tại construction time (trước khi configure), field sẽ trở vào `NullLogger` cũ mãi mãi, kể cả sau khi `Log.Logger` được replace.

Với **property**, mỗi lần gọi `Logger.Information(...)` sẽ evaluate `=> Log.Logger` tại thời điểm đó, luôn trở vào logger hiện tại.

8.1.3 ConnectAsync - Flow chi tiết

Step 1: Guard checks

- └─ if (_disposed) throw ObjectDisposedException
- └─ if (already Connected/Connecting) return

Step 2: State transition

- └─ ConnectionState = Connecting

Step 3: Create ApplicationConfiguration

- └─ ApplicationName, URI, certificates
- └─ Transport quotas (message size, timeout)
- └─ Validate certs, create self-signed if needed

Step 4: Endpoint discovery with retry

- └─ SelectEndpointAsync(endpointUrl, useSecurity)
- └─ Retry 3 times với exponential backoff
- └─ Throw if all retries fail

Step 5: Session.Create()

- └─ Pass endpoint, clientCertificate, sessionTimeout=60s
- └─ Returns active OPC UA Session

Step 6: Wire events

- └─ _session.KeepAlive += OnKeepAlive
- └─ _session.Notification += OnNotification
- └─ _session.PublishError += OnPublishError

Step 7: State = Connected

- └─ ConnectionStateChanged event raised

Step 8: Create subscriptions

- └─ foreach SubscriptionGroup in _device.SubscriptionGroups where Enabled
 - └─ CreateSubscriptionAsync(group)

Bước 4 - Retry discovery: OPC UA discovery protocol là UDP-based và đôi khi unreliable trên một số networks. Retry 3 lần với delay giữa các lần giúp handle transient network issues.

Bước 6 - Wire events AFTER session created: Nếu wire events trước khi session tồn tại, event handlers có thể fire với null session → NullReferenceException.

8.1.4 DisconnectAsync - Flow chi tiết

```
Step 1: _isDisconnecting = true
        (prevents reconnect from triggering during cleanup)

Step 2: Dispose _reconnectHandler
        (stop SDK-level reconnect)

Step 3: StopAutoReconnect()
        (cancel our custom reconnect loop)

Step 4: Unwire events FIRST
├─ _session.KeepAlive -= OnKeepAlive
├─ _session.Notification -= OnNotification
└─ _session.PublishError -= OnPublishError

Step 5: Delete all subscriptions
        (graceful server-side cleanup)

Step 6: session.CloseAsync() with 5s timeout
        (tell server we're closing)

Step 7: session.Dispose()

Step 8: _session = null

Step 9: ConnectionState = Disconnected

Step 10: _isDisconnecting = false
```

Tại sao unwire events **TRƯỚC** khi close session?

Khi session đang được close, các KeepAlive failures hoặc network errors có thể trigger event handlers. Nếu event handler vẫn còn wired, chúng có thể try to reconnect trong khi chúng ta đang intentionally disconnecting → race condition.

5 second timeout cho CloseAsync: Nếu network đã drop, CloseAsync sẽ hang. Timeout đảm bảo disconnect không block indefinitely.

8.1.5 CRITICAL BUG: NodeId.Parse vs new NodeId

Đây là một trong những bug subtle nhất trong OPC UA programming với C# SDK.

```
// CORRECT - Numeric NodeId
var nodeId = NodeId.Parse("i=85");
// Result: NodeId { NamespaceIndex=0, IdType=Numeric, Identifier=85 }

// WRONG - String NodeId
var nodeId = new NodeId("i=85");
// Result: NodeId { NamespaceIndex=0, IdType=String, Identifier="i=85" }
```

Tại sao điều này quan trọng trong OPC UA protocol?

OPC UA NodeId có hai component: 1. **IdType**: Numeric, String, Guid, ByteString 2.

Identifier: Value tương ứng với type

"i=85" là **text representation** của một Numeric NodeId: - **i=** prefix có nghĩa là Numeric - **85** là identifier value

`NodeId.Parse("i=85")` **hiểu** text representation này và tạo Numeric NodeId với identifier=85.

`new NodeId("i=85")` **không parse** - nó tạo String NodeId với identifier là toàn bộ chuỗi "i=85".

Khi gửi request đến OPC UA server: - Server tìm node với Numeric identifier = 85 → FOUND (ví dụ: Objects folder) - Server tìm node với String identifier = "i=85" → NOT FOUND → BadNodeIdUnknown error

Bug này đặc biệt khó debug vì code compile và run không có error, chỉ receive BadNodeIdUnknown từ server - không biết tại sao.

Namespace index cũng quan trọng:

```
// NodeId với namespace 2
NodeId.Parse("ns=2;i=1001") // Correct
new NodeId("ns=2;i=1001")   // Wrong - String NodeId với identifier "ns=2;i=1001"
```

Luôn dùng `NodeId.Parse()` khi parse từ string representation.

8.1.6 KeepAlive Settling Period

```
private void OnKeepAlive(ISession session, KeepAliveEventArgs e)
{
    // Ignore KeepAlive events right after reconnect completes
    if ((DateTime.UtcNow - _lastReconnectCompleteTime).TotalMilliseconds < KeepAliveSettlingPeriodMs)
        return;

    if (e.Status.IsGood()) return; // Normal keepalive, all good

    if (_isDisconnecting) return; // Intentional disconnect, don't reconnect

    // KeepAlive failed → trigger reconnect
    StartAutoReconnect();
}
```

Tại sao cần 2000ms settling period?

OPC UA Session reconnect là hai-layer process: 1. **Layer 1**: SDK `SessionReconnectHandler` tự động reconnect Secure Channel 2. **Layer 2**: Sau khi Secure Channel up, application cần re-create Session

Khi Session reconnect hoàn thành và `_lastReconnectCompleteTime` được set, server vẫn có thể gửi KeepAlive responses cho **Secure Channel cũ** đang trong queue. Những responses này có thể có status là bad (vì channel đã close).

Nếu chúng ta process những events này ngay sau reconnect, chúng ta sẽ trigger reconnect lại một lần nữa → reconnect loop vô tận.

Settling period 2000ms = ignore events trong 2 giây sau reconnect → đủ thời gian cho queue cũ drain.

8.2 PlcManager - Quản lý nhiều PlcConnection

8.2.1 Kiến trúc

`PlcManager` là **facade** và **coordinator** cho nhiều `PlcConnection`. Application code không tương tác trực tiếp với `PlcConnection` - chỉ thông qua `PlcManager`.

```
public class PlcManager : IPlcManager, IDisposable
{
    private readonly ConcurrentDictionary<string, IPlcConnection> _connections = new();
    private readonly IPlcConnectionFactory _connectionFactory;
    private readonly IDataCacheService _dataCache;

    public event EventHandler<PlcConnectionStateChangedEventArgs>? ConnectionStateChanged;
    public event EventHandler<TagValueChangedEventArgs>? TagValueChanged;
}
```

ConcurrentDictionary<string, IPlcConnection>: Key là PlcId, value là connection. Thread-safe vì multiple threads có thể: - UI thread: disconnect một PLC - Timer thread: reconnect - API thread: đọc connection state

8.2.2 AddPlcAsync - Wire Events Pattern

```
public async Task AddPlcAsync(PlcDevice device)
{
    // 1. Factory creates connection
    var connection = _connectionFactory.Create(device);

    // 2. Wire events BEFORE adding to dictionary
    connection.ConnectionStateChanged += OnConnectionStateChanged;
    connection.TagValueChanged += OnTagValueChanged;

    // 3. Add to dictionary
    _connections[device.PlcId] = connection;

    // 4. Auto-connect if enabled
    if (device.AutoConnect)
        await connection.ConnectAsync();
}
```

Tại sao wire events trước khi add vào dictionary? Tránh race condition: nếu add trước rồi wire events sau, có thể có thread khác đọc từ dictionary và access connection chưa có event handlers.

8.2.3 RemovePlcAsync - Cleanup Order

```
public async Task RemovePlcAsync(string plcId)
{
    if (!_connections.TryRemove(plcId, out var connection))
        return;

    // CRITICAL: Unsubscribe events BEFORE dispose
    connection.ConnectionStateChanged -= OnConnectionStateChanged;
    connection.TagValueChanged -= OnTagValueChanged;

    await connection.DisconnectAsync();
    connection.Dispose();
}
```

Tại sao unsubscribe events **TRƯỚC** khi dispose?

Dispose sẽ trigger DisconnectAsync, có thể raise ConnectionStateChanged event. Nếu chúng ta đã remove connection khỏi dictionary nhưng chưa unsubscribe events, handler sẽ fire với một connection không còn tồn tại trong _connections → zombie event handler.

Zombie handler có thể: - Gây null reference nếu handler assumes connection còn trong dictionary - Update DataCache với stale data - Log confusing state changes

Thứ tự đúng: TryRemove khỏi dict → Unsubscribe events → Disconnect → Dispose.

8.2.4 ConnectAllAsync - Parallel Connection

```
public async Task ConnectAllAsync()
{
    var tasks = _connections.Values
        .Where(c => c.Device.Enabled && c.State != PlcConnectionState.Connected)
        .Select(c => c.ConnectAsync())
        .ToList();

    await Task.WhenAll(tasks);
}
```

`Task.WhenAll(tasks)` : Chạy tất cả connect operations **song song**. Nếu có 10 PLC, tổng thời gian là max(thời gian connect 1 PLC) thay vì sum.

So sánh: - Sequential: 10 PLCs x 3s = 30s startup time - Parallel (Task.WhenAll): max(3s) = 3s startup time

Điều này đặc biệt quan trọng khi startup - user không muốn đợi 30 giây trước khi UI responsive.

8.2.5 Two-Layer Reconnect Strategy

OPC UA reconnect có hai layer:

Layer 1 - SessionReconnectHandler (SDK):

```
Secure Channel drop detected
→ SessionReconnectHandler starts
→ Tries to restore Secure Channel
→ If successful: session.Reconnect() restores session state
→ SubscriptionTransferring event: re-transfer monitored items
```

Layer 2 - Custom reconnect loop:

```
KeepAlive failure detected (Layer 1 failed or timed out)
→ StartAutoReconnect() called
→ Loop với exponential backoff
→ ConnectAsync() creates brand new Session
→ OnConnected: re-create all subscriptions
```

Tại sao cần hai layer?

Layer 1 (SessionReconnectHandler) xử lý **transient failures** - brief network blip, server restart nhanh. SDK có thể restore session với subscriptions nguyên vẹn.

Layer 2 xử lý **prolonged failures** - server down lâu, Layer 1 timeout. Khi đó phải tạo session mới và re-subscribe.

Kết hợp hai layer cho phép fast recovery (Layer 1) cho common case, và reliable recovery (Layer 2) cho worse cases.

8.2.6 OnTagValueChanged - DataCache Update

```
private void OnTagValueChanged(object? sender, TagValueChangedEventArgs e)
{
    // Update cache
    _dataCache.UpdateTagValue(e.TagId, new TagValue
    {
        Value = e.Value,
        Quality = e.Quality,
        Timestamp = e.Timestamp
    });

    // Forward event to PlcManager subscribers
    TagValueChanged?.Invoke(this, e);
}
```

Flow khi tag thay đổi: 1. OPC UA server gửi Notification 2. SDK fires

`_session.Notification` event 3. `PlcConnection.OnNotification` processes notification, fires `TagValueChanged` 4. `PlcManager.OnTagValueChanged` receives event 5. Update DataCache (để REST API serve latest value) 6. Forward event (ViewModel có thể subscribe để update UI)

DataCache và UI update xảy ra tại cùng một thời điểm từ cùng một event. DataCache update là thread-safe (ConcurrentDictionary). UI update phải marshal sang UI thread qua Dispatcher.

8.3 Tổng kết kiến trúc OPC UA Layer

```
PlcManager
├─ PlcConnection (PLC-A)
│   └─ OPC UA Session
│       ├── Subscription (Group-1)
│       │   ├── MonitoredItem (Tag-1)
│       │   └── MonitoredItem (Tag-2)
│       └── Subscription (Group-2)
│           └── MonitoredItem (Tag-3)
│   └─ Events: StateChanged, TagValueChanged
└─ PlcConnection (PLC-B)
    └─ ...

DataCacheService
└─ ConcurrentDictionary<tagId, TagValue>
    (updated by PlcManager.OnTagValueChanged)

REST API
└─ GET /api/tags/{id} → reads from DataCacheService
```

Kiến trúc phân lớp rõ ràng: - **PlcConnection** : biết về OPC UA protocol - **PlcManager** : biết về multiple PLCs, routing events - **DataCacheService** : biết về caching - REST API: biết về HTTP - ViewModel: biết về UI

Mỗi layer chỉ communicate với layer liền kề - loose coupling, dễ test và maintain.